

Teil 1: Grundlagen und erste Schritte	6
Teil 2: Ownership und zusammengesetzte Datentypen	29
Teil 3: Enums, Pattern Matching und Collections	53
Teil 4: Fehlerbehandlung und Code-Organisation	71
Teil 5: Weiterführende Konzepte (Einführung)	87

Okay, hier ist ein Vorschlag für ein Inhaltsverzeichnis für Rust-Anfänger, aufgeteilt in Teile und Kapitel. Dieses Verzeichnis deckt die grundlegenden Konzepte ab, die man für den Einstieg in Rust benötigt.

Rust für Anfänger: Ein Inhaltsverzeichnis

Teil 1: Grundlagen und erste Schritte

- **Kapitel 1: Einführung und Installation**
 - Was ist Rust und warum sollte man es lernen?
 - Installation von Rust (rustup, cargo)
 - Dein erstes Rust-Programm: "Hallo, Welt!"
 - Kompilieren und Ausführen mit `cargo`
- **Kapitel 2: Grundlegende Syntax und Konzepte**
 - Anatomie eines Rust-Programms
 - Kommentare
 - Variablen und Mutabilität (`let`, `mut`)
 - Shadowing (Variablen überschatten)
- **Kapitel 3: Datentypen I - Skalare Typen**
 - Ganzzahlen (Integers: `i32`, `u64`, etc.)
 - Gleitkommazahlen (Floats: `f32`, `f64`)
 - Boolesche Werte (`bool`)
 - Zeichen (`char`)
 - Grundlegende Operatoren (+, -, *, /, %)
- **Kapitel 4: Funktionen**
 - Funktionen definieren und aufrufen
 - Parameter und Argumente
 - Funktionen mit Rückgabewerten
 - Anweisungen vs. Ausdrücke
- **Kapitel 5: Kontrollfluss I - Bedingungen**
 - `if/else`-Ausdrücke
 - `else if`
 - Bedingungen in `let`-Anweisungen

Teil 2: Ownership und zusammengesetzte Datentypen

- **Kapitel 6: Das Ownership-Konzept**

- Was ist Ownership? (Stack vs. Heap kurz erklärt)
- Die Ownership-Regeln
- Move-Semantik
- Clone-Semantik (`.clone()`)
- Ownership und Funktionen
- **Kapitel 7: Referenzen und Borrowing (Leihen)**
 - Das Problem mit Ownership und Datenweitergabe
 - Unveränderliche Referenzen (`&`)
 - Veränderliche Referenzen (`&mut`)
 - Die Borrowing-Regeln (nur eine mutable ODER beliebig viele immutable Referenzen)
 - Dangling References (ungültige Referenzen vermeiden)
- **Kapitel 8: Slices**
 - Was sind Slices?
 - String-Slices (`&str`)
 - Andere Slices (z.B. von Arrays)
- **Kapitel 9: Datentypen II - Zusammengesetzte Typen**
 - Tupel
 - Arrays (feste Größe)
- **Kapitel 10: Structs - Eigene Datentypen erstellen**
 - Structs definieren und instanziiieren
 - Zugriff auf Struct-Felder
 - Tuple Structs
 - Methoden für Structs definieren (`impl`)
 - Assoziierte Funktionen (wie Konstruktoren)

Teil 3: Enums, Pattern Matching und Collections

- **Kapitel 11: Enums und Pattern Matching**
 - Enums definieren (Aufzählungen)
 - Daten mit Enum-Varianten verknüpfen
 - Der `Option<T>`-Enum
 - Der `match`-Kontrollflussoperator
 - Muster, die Werte binden
 - `if let` für einfache Fälle
- **Kapitel 12: Kontrollfluss II - Schleifen**
 - Endlosschleife mit `loop`

- Bedingte Schleifen mit `while`
- Schleifen über Collections mit `for`
- **Kapitel 13: Collections I - Vektoren**
 - Dynamische Listen mit `Vec<T>`
 - Vektoren erstellen, aktualisieren und lesen
 - Iterieren über Vektor-Elemente
 - Enums zum Speichern verschiedener Typen in Vektoren nutzen
- **Kapitel 14: Collections II - Strings**
 - Rusts Umgang mit Text (`String` vs. `&str`)
 - Strings erstellen und aktualisieren
 - UTF-8 und Indizierung von Strings (die Herausforderungen)
 - Methoden zur String-Manipulation
 - Iterieren über Strings (Zeichen, Bytes)
- **Kapitel 15: Collections III - Hash Maps**
 - Schlüssel-Wert-Paare mit `HashMap<K, V>`
 - Hash Maps erstellen, darauf zugreifen und aktualisieren
 - Ownership bei Hash Maps

Teil 4: Fehlerbehandlung und Code-Organisation

- **Kapitel 16: Fehlerbehandlung I - Unbehebbarer Fehler mit `panic!`**
 - Wann `panic!` verwenden?
 - Backtraces bei Panics
- **Kapitel 17: Fehlerbehandlung II - Behebbarer Fehler mit `Result<T, E>`**
 - Der `Result<T, E>`-Enum
 - Umgang mit `Result`-Werten (`match`, `unwrap`, `expect`)
 - Fehler weitergeben mit dem `?`-Operator
 - Wann `Result` statt `panic!` verwenden?
- **Kapitel 18: Module und das Modulsystem**
 - Code in Module aufteilen (`mod`)
 - Pfade zu Modulen (`use`, `super`, `self`)
 - Sichtbarkeit kontrollieren (`pub`)
 - Dateien für Module verwenden
- **Kapitel 19: Crates und Cargo**
 - Was ist ein Crate? (Binary vs. Library)
 - Pakete und Crates
 - Externe Crates von crates.io verwenden (`Cargo.toml`)

- `cargo build`, `cargo run`, `cargo check`

Teil 5: Weiterführende Konzepte (Einführung)

- **Kapitel 20: Generics - Wiederverwendbaren Code schreiben**
 - Generische Typen in Funktionsdefinitionen
 - Generische Typen in Struct- und Enum-Definitionen
- **Kapitel 21: Traits - Gemeinsames Verhalten definieren**
 - Was sind Traits? (Ähnlich zu Interfaces)
 - Traits definieren und implementieren
 - Traits als Parameter
 - Trait Bounds
- **Kapitel 22: Lifetimes - Gültigkeitsdauern verstehen (Grundlagen)**
 - Warum Lifetimes gebraucht werden (bei Referenzen)
 - Lifetime-Annotationen in Funktionssignaturen
 - Grundlegende Regeln und Inferenz
- **Kapitel 23: Automatisierte Tests**
 - Testfunktionen schreiben (`#[test]`)
 - `assert!`, `assert_eq!`, `assert_ne!`
 - Tests mit `cargo test` ausführen
 - Tests für `panic!`
- **Kapitel 24: Ein kleines Projekt**
 - Anwendung der gelernten Konzepte in einem einfachen Kommandozeilenprogramm (z.B. Ratespiel, einfacher Taschenrechner)

Teil 1: Grundlagen und erste Schritte

Sehr geehrter Lernender,

Teil 1: Grundlagen und erste Schritte

Dieser erste Teil legt das Fundament für Ihr Verständnis von Rust. Wir werden die Sprache selbst kennenlernen, die notwendigen Werkzeuge installieren, unser erstes Programm schreiben und grundlegende syntaktische sowie konzeptionelle Elemente erarbeiten.

Kapitel 1: Einführung und Installation

In diesem ersten Kapitel geht es darum, ein grundlegendes Verständnis dafür zu entwickeln, was Rust ist, warum es eine relevante und oft vorteilhafte Wahl darstellt, und wie wir unsere Entwicklungsumgebung einrichten, um mit dem Programmieren beginnen zu können.

- **1.1 Was ist Rust und warum sollte man es lernen?**

1. Was ist Rust?

Rust ist eine moderne Systemprogrammiersprache, die sich auf drei Kernziele konzentriert: Sicherheit, Geschwindigkeit und Parallelität. Lassen Sie uns diese Begriffe genauer betrachten:

- **Systemprogrammiersprache:** Dies bedeutet, dass Rust darauf ausgelegt ist, Software zu schreiben, die sehr nah an der Hardware operiert. Denken Sie an Betriebssysteme, Webbrowser-Engines, Spiele-Engines, Kommandozeilenwerkzeuge oder eingebettete Systeme (wie Mikrocontroller). Solche Software erfordert oft eine sehr feingranulare Kontrolle über Speicherressourcen und hohe Performance. Traditionell wurde dieser Bereich von Sprachen wie C und C++ dominiert.
- **Sicherheit:** Eines der herausragendsten Merkmale von Rust ist seine Speicher- und Threadsicherheit – und das zur *Kompilierzeit*. Viele Fehler, die in C oder C++ erst zur Laufzeit auftreten und zu Abstürzen oder Sicherheitslücken führen können (z.B. Null-Pointer-Dereferenzierungen, Pufferüberläufe, Datenwettläufe in nebenläufigem Code), werden vom Rust-Compiler bereits während des Kompilierens erkannt und

verhindert. Dies wird hauptsächlich durch das **Ownership- und Borrowing-System** erreicht, auf das wir in Teil 2 detailliert eingehen werden. Es ist ein zentrales Konzept, das Rust von vielen anderen Sprachen unterscheidet. Man kann es sich vorstellen wie einen extrem strengen, aber fairen Bauleiter (der Compiler), der sicherstellt, dass alle Pläne (der Code) absolut sicher sind, bevor überhaupt mit dem Bau (der Ausführung) begonnen wird.

- **Geschwindigkeit:** Rust ist eine kompilierte Sprache, die direkt in Maschinencode übersetzt wird. Sie benötigt keine virtuelle Maschine oder einen Interpreter (wie z.B. Java oder Python). Dadurch erreicht Rust eine Performance, die mit der von C und C++ vergleichbar ist. Wichtig ist hierbei, dass Rust diese Geschwindigkeit *ohne* einen Garbage Collector erreicht. Ein Garbage Collector ist ein Mechanismus in vielen Sprachen (z.B. Java, C#, Go, Python), der automatisch ungenutzten Speicher bereinigt. Dies bringt zwar Komfort, kann aber zu unvorhersehbaren Pausen in der Programmausführung führen ("Stop-the-World"-Pausen), was für systemnahe oder echtzeitkritische Anwendungen oft problematisch ist. Rust verwaltet den Speicher durch das bereits erwähnte Ownership-System zur Kompilierzeit, was zu einer deterministischeren Performance führt.
- **Parallelität (Concurrency):** Modernes Programmieren erfordert oft die Ausführung mehrerer Aufgaben gleichzeitig, um die Leistungsfähigkeit moderner Mehrkernprozessoren auszunutzen. Parallelität ist jedoch notorisch fehleranfällig, insbesondere durch sogenannte *Data Races* (Datenwettläufe), bei denen mehrere Threads gleichzeitig auf dieselben Daten zugreifen, mindestens einer davon schreibend, ohne geeignete Synchronisation. Rusts Ownership- und Typsystem helfen auch hier enorm, indem sie Data Races bereits zur Kompilierzeit verhindern. Rust macht "fearless concurrency" (furchtlose Parallelität) zu einem seiner Verkaufsargumente.

2. Warum Rust lernen?

Es gibt viele gute Gründe, sich mit Rust zu beschäftigen:

- **Zuverlässigkeit:** Durch die strengen Kompilierzeit-Checks entstehen robustere Programme mit weniger Laufzeitfehlern, insbesondere im Bereich der Speicherverwaltung.
- **Performance:** Wenn Sie Anwendungen schreiben müssen, die maximale Geschwindigkeit und Effizienz erfordern, ist Rust eine ausgezeichnete Wahl.

- **Moderne Features:** Rust bietet viele Annehmlichkeiten moderner Sprachen, wie ein starkes Typsystem mit Typinferenz, Pattern Matching, Generics, Traits (ähnlich Interfaces), algebraische Datentypen (Enums), ein eingebautes Build-System und Paketmanagement (Cargo) und vieles mehr.
- **WebAssembly-Unterstützung:** Rust hat erstklassige Unterstützung für WebAssembly (Wasm), was es ermöglicht, Rust-Code mit nahezu nativer Geschwindigkeit direkt im Webbrowser auszuführen.
- **Wachsende Community und Ökosystem:** Rust hat eine sehr aktive, freundliche und hilfsbereite Community. Das Ökosystem an Bibliotheken (genannt "Crates") wächst stetig.
- **Hohe Nachfrage:** Entwickler mit Rust-Kenntnissen sind zunehmend gefragt, da immer mehr Unternehmen Rust für kritische Systeme einsetzen.
- **Lernwert:** Selbst wenn Sie Rust nicht täglich einsetzen, lehrt das Erlernen des Ownership-Konzepts Sie viel über Speicherverwaltung und Ressourcensicherheit, was auch Ihr Verständnis und Ihren Stil in anderen Programmiersprachen verbessern kann.

- **1.2 Installation von Rust (rustup, cargo)**

Um mit Rust entwickeln zu können, benötigen wir den Rust-Compiler und zugehörige Werkzeuge. Der empfohlene Weg, Rust zu installieren und zu verwalten, ist die Verwendung von `rustup`.

1. `rustup`: Dies ist der offizielle Rust-Installer und Versionsmanager. Man kann ihn sich wie einen Verwalter für Ihre Rust-Installationen vorstellen. Warum ist das nützlich?

- **Einfache Installation:** `rustup` lädt den Compiler (`rustc`), den Paketmanager (`cargo`), die Standardbibliothek und die Dokumentation herunter und installiert alles korrekt für Ihr Betriebssystem (Windows, macOS, Linux).
- **Versionsmanagement:** Rust veröffentlicht alle sechs Wochen eine neue stabile Version. `rustup` macht es sehr einfach, zwischen verschiedenen Versionen (stable, beta, nightly) zu wechseln oder Ihre Installation auf den neuesten Stand zu bringen (mit `rustup update`). Sie können sogar unterschiedliche Versionen für verschiedene Projekte verwenden.
- **Toolchain-Management:** Rust besteht aus mehreren Komponenten (Compiler, Standardbibliothek, etc.), die zusammen als "Toolchain" bezeichnet werden. `rustup` verwaltet diese Toolchains und ermöglicht auch die Installation

zusätzlicher Komponenten (z.B. für Cross-Compilation zu anderen Zielplattformen).

2. Die Installation erfolgt typischerweise durch den Besuch der offiziellen Rust-Website (<https://www.rust-lang.org/>) und das Befolgen der dortigen Anweisungen. Meistens beinhaltet dies das Herunterladen und Ausführen eines kleinen Skripts, das dann `rustup` installiert, welches wiederum die aktuelle stabile Rust-Toolchain herunterlädt und einrichtet. Nach der Installation sollten die Befehle `rustc --version` und `cargo --version` in Ihrem Terminal oder Ihrer Eingabeaufforderung funktionieren und die installierten Versionen anzeigen.
3. **cargo:** Cargo ist Rusts eingebautes Build-System und Paketmanager. Es ist eines der am meisten geschätzten Werkzeuge im Rust-Ökosystem. Cargo übernimmt viele Aufgaben für Sie:
 - **Projektmanagement:** Mit `cargo new <projektname>` oder `cargo init` erstellen Sie die Grundstruktur eines neuen Rust-Projekts.
 - **Abhängigkeitsmanagement:** Rust-Bibliotheken werden "Crates" genannt und auf crates.io, dem offiziellen Paket-Repository, veröffentlicht. Sie deklarieren die benötigten externen Crates in einer Datei namens `Cargo.toml` in Ihrem Projekt, und Cargo lädt sie automatisch herunter und kompiliert sie.
 - **Kompilieren:** `cargo build` kompiliert Ihren Code und alle Abhängigkeiten.
 - **Ausführen:** `cargo run` kompiliert (falls nötig) und führt Ihr Programm aus.
 - **Testen:** `cargo test` führt alle Tests in Ihrem Projekt aus.
 - **Dokumentation:** `cargo doc` erstellt eine lokale HTML-Dokumentation für Ihr Projekt und seine Abhängigkeiten.
 - **Überprüfen:** `cargo check` überprüft Ihren Code schnell auf Fehler, ohne eine ausführbare Datei zu erzeugen, was oft schneller ist als `cargo build`.
4. Cargo vereinfacht den Entwicklungsprozess enorm und ist ein integraler Bestandteil des Rust-Erlebnisses.

● 1.3 Dein erstes Rust-Programm: "Hallo, Welt!"

Traditionell beginnt man das Erlernen einer neuen Programmiersprache mit einem Programm, das "Hallo, Welt!" (oder "Hello, world!") ausgibt. Lassen Sie uns das auch in Rust tun.

1. **Projekt erstellen:** Öffnen Sie Ihr Terminal oder Ihre Eingabeaufforderung, navigieren Sie in ein Verzeichnis, in dem Sie Ihre

Projekte speichern möchten, und führen Sie folgenden Befehl aus:

2. Bash

```
cargo new hello_welt
```

3.

4. Cargo erstellt nun ein neues Verzeichnis namens `hello_welt` mit folgender Struktur:

```
hello_welt/  
├── Cargo.toml  
└── src/  
    └── main.rs
```

5.

- `Cargo.toml`: Die Konfigurationsdatei für Cargo (enthält Metadaten, Abhängigkeiten etc.). Wir schauen später genauer hinein.
- `src/main.rs`: Hier befindet sich unser Quellcode. Cargo hat bereits eine einfache `main.rs`-Datei für uns generiert.

6. **Code ansehen:** Öffnen Sie die Datei `src/main.rs` in einem Texteditor oder einer IDE. Sie sollte folgenden Inhalt haben:

7. Rust

```
fn main() {  
    println!("Hallo, Welt!");  
}
```

8.

9. Lassen Sie uns diesen Code Zeile für Zeile analysieren:

- `fn main() { ... }`: Dies definiert eine **Funktion** namens `main`. Die Funktion `main` ist besonders wichtig, denn sie ist der **Einstiegspunkt** für jedes ausführbare Rust-Programm. Wenn Sie Ihr Programm starten, ist der Code innerhalb der geschweiften Klammern `{ }` der Funktion `main` der erste Code, der ausgeführt wird. `fn` ist das Schlüsselwort zur Definition einer Funktion. Die leeren Klammern `()` bedeuten, dass diese Funktion keine **Parameter** entgegennimmt.

- `println!("Hallo, Welt!");`: Diese Zeile gibt den Text "Hallo, Welt!" auf der Konsole aus. Hier gibt es ein paar interessante Details:
 - `println!` ist kein gewöhnlicher Funktionsaufruf, sondern ein **Makroaufruf**. In Rust erkennen Sie Makros am Ausrufezeichen `!` am Ende des Namens. Makros sind eine Art von Metaprogrammierung in Rust – Code, der anderen Code schreibt. `println!` ist ein sehr nützliches Makro aus der Standardbibliothek, das Text formatiert und auf der Standardausgabe (normalerweise Ihr Terminal) mit einem abschließenden Zeilenumbruch ausgibt. Wir werden später noch mehr über Makros lernen, für den Moment merken wir uns: `println!` zur Ausgabe mit Zeilenumbruch.
 - `"Hallo, Welt!"` ist ein **String-Literal**. Es repräsentiert eine Zeichenkette, also eine Sequenz von Textzeichen. In Rust werden String-Literale in doppelte Anführungszeichen eingeschlossen.
 - Die Zeile endet mit einem **Semikolon** `;`. In Rust beenden die meisten **Anweisungen** (Statements) mit einem Semikolon. Es signalisiert das Ende der Anweisung.

- **1.4 Kompilieren und Ausführen mit `cargo`**

Nachdem wir unser erstes Programm geschrieben (bzw. von `cargo new` generieren lassen) haben, wollen wir es nun ausführen.

1. **Navigieren Sie ins Projektverzeichnis:** Stellen Sie sicher, dass Sie sich in Ihrem Terminal im Verzeichnis `hallo_welt` befinden (also dem Verzeichnis, das `Cargo.toml` enthält).

2. Bash

`cd hallo_welt`

- 3.

- 4.

5. **Kompilieren und Ausführen (der einfachste Weg):**

6. Bash

`cargo run`

- 7.

8. Dieser Befehl macht zwei Dinge:

- **Kompilieren:** Cargo ruft den Rust-Compiler `rustc` auf, um Ihren Quellcode (`src/main.rs`) in ausführbaren Maschinencode zu übersetzen. Das Ergebnis wird standardmäßig im Verzeichnis `target/debug/` abgelegt (z.B. als `hallo_welt` oder `hallo_welt.exe` unter Windows). Da dies das erste Mal ist, werden auch alle eventuell benötigten Teile der Standardbibliothek mitkompiliert. Zukünftige Kompilervorgänge sind oft schneller, da nur geänderte Teile neu kompiliert werden.
- **Ausführen:** Nach erfolgreicher Kompilierung führt Cargo die erstellte ausführbare Datei aus.

9. Sie sollten nun die folgende Ausgabe in Ihrem Terminal sehen:

Hallo, Welt!

10.

11.

12. **Nur Kompilieren:** Sie können den Code auch nur kompilieren, ohne ihn sofort auszuführen:

13. Bash

cargo build

14.

15. Dieser Befehl erzeugt die ausführbare Datei in `target/debug/`. Sie könnten sie dann manuell ausführen (z.B. unter Linux/macOS mit `./target/debug/hallo_welt` oder unter Windows mit `.\target\debug\hallo_welt.exe`). Normalerweise ist `cargo run` aber bequemer.

16. **Schnell überprüfen (ohne Ausführen):**

17. Bash

cargo check

18.

19. Dieser Befehl führt eine schnelle Überprüfung Ihres Codes durch den Compiler durch, um sicherzustellen, dass er syntaktisch korrekt ist und die Typprüfungen besteht. Er erzeugt jedoch keine ausführbare Datei.

Dies ist oft deutlich schneller als `cargo build` und nützlich während der Entwicklung, um schnell Feedback zu bekommen, ob der Code (zumindest syntaktisch) in Ordnung ist.

Herzlichen Glückwunsch! Sie haben soeben Ihr erstes Rust-Programm erfolgreich kompiliert und ausgeführt und dabei die grundlegenden Werkzeuge `rustup` und `cargo` kennengelernt.

Kapitel 2: Grundlegende Syntax und Konzepte

Nachdem wir nun wissen, wie man ein Rust-Projekt aufsetzt und ausführt, wollen wir uns die grundlegende Struktur und einige Kernkonzepte der Sprache genauer ansehen.

- **2.1 Anatomie eines Rust-Programms**

Wir haben bereits die `main`-Funktion gesehen. Ein Rust-Programm besteht im Wesentlichen aus einer Sammlung von **Funktionen**, **Modulen**, **Structs**, **Enums**, **Traits** und anderen Elementen. Für den Anfang konzentrieren wir uns auf Funktionen.

- Rust

```
// Dies ist ein Kommentar
```

```
// Die main-Funktion ist der Einstiegspunkt
```

```
fn main() {  
    // Anweisungen innerhalb der Funktion  
    println!("Eine Anweisung.");
```

```
    // Funktionsaufruf einer anderen Funktion  
    eine_andere_funktion();  
} // Ende des main-Funktionsblocks
```

```
// Definition einer anderen Funktion
```

```
fn eine_andere_funktion() {  
    println!("Aufruf aus einer anderen Funktion.");  
} // Ende des Funktionsblocks
```

-
- Wichtige Elemente der Anatomie sind:
 - `fn` **Schlüsselwort:** Leitet die Definition einer Funktion ein.

- **Funktionsname:** Folgt `fn`, üblicherweise in `snake_case` (Kleinbuchstaben mit Unterstrichen zur Worttrennung).
- **Parameterliste:** In runden Klammern `()`. Kann leer sein oder Parameter enthalten.
- **Funktionskörper:** Eingeschlossen in geschweifte Klammern `{ }`. Enthält die Anweisungen und Ausdrücke, die die Funktion ausführt.
- **Anweisungen (Statements):** Aktionen, die ausgeführt werden. Die meisten enden mit einem Semikolon `;`. Beispiel: `println!(...);`.
- **Ausdrücke (Expressions):** Werten zu einem Wert aus. Beispiel: `5 + 6` ist ein Ausdruck, der zu `11` ausgewertet wird. Wir werden später sehen, dass viele Konstrukte in Rust, wie `if` oder Funktionskörper selbst, Ausdrücke sind.
- **Kommentare:** Werden vom Compiler ignoriert, dienen der Erklärung des Codes für Menschen.

● 2.2 Kommentare

Kommentare sind essentiell, um Code verständlich zu machen. Rust kennt zwei Hauptarten von Kommentaren:

- **Zeilenkommentare:** Beginnen mit `//` und gehen bis zum Ende der Zeile. Alles nach `//` wird vom Compiler ignoriert.

○ Rust

`// Dies ist ein ganzer Zeilenkommentar.`

`let x = 5; // Dies ist ein Kommentar am Ende einer Zeile.`

-
-
- **Blockkommentare:** Beginnen mit `/*` und enden mit `*/`. Sie können sich über mehrere Zeilen erstrecken und auch mitten in einer Zeile verwendet werden (obwohl letzteres selten und oft unleserlich ist).

○ Rust

```
/*
Dies ist ein
mehrzeiliger
Blockkommentar.
*/
```

`let y = /* hier könnte ein Kommentar stehen */ 10;`

○

- Blockkommentare können nützlich sein, um vorübergehend größere Codeabschnitte auszukommentieren, da sie auch verschachtelt werden können (obwohl Zeilenkommentare oft einfacher zu handhaben sind).
- **Dokumentationskommentare:** Es gibt spezielle Kommentare (`///` für Elemente, `//!` für umschließende Module/Crates), die von `cargo doc` verwendet werden, um automatisch Dokumentation zu generieren. Diese werden wir später behandeln, aber es ist gut zu wissen, dass sie existieren.

● 2.3 Variablen und Mutabilität (`let`, `mut`)

Variablen ermöglichen es uns, Werte zu speichern und ihnen einen Namen zu geben, um sie später wiederverwenden zu können.

- **Deklaration mit `let`:** In Rust deklarieren wir Variablen mit dem Schlüsselwort `let`.

- Rust

```
fn main() {
    let x = 5; // Deklariert eine Variable 'x' und bindet den Wert 5 daran.
    println!("Der Wert von x ist: {}", x);

    let name = "Rust"; // Variablen können auch andere Typen haben, hier ein String-Literal.
    println!("Hallo, {}!", name);
}
```

- Rust ist eine **statisch typisierte** Sprache, was bedeutet, dass der Typ jeder Variablen zur Kompilierzeit bekannt sein muss. Oft kann der Compiler den Typ jedoch aus dem zugewiesenen Wert **ableiten** (Typinferenz). Im Beispiel `let x = 5;` erkennt der Compiler, dass `5` eine Ganzzahl ist und weist `x` standardmäßig den Typ `i32` (32-Bit-Integer) zu. Bei `let name = "Rust";` wird der Typ `&str` (ein String-Slice) abgeleitet. Wir könnten den Typ auch explizit angeben: `let x: i32 = 5;`.
- **Immutabilität (Unveränderlichkeit) per Default:** Eine der wichtigsten Eigenschaften von Rust ist, dass Variablen standardmäßig **immutable** (unveränderlich) sind. Das bedeutet, nachdem einer Variablen mit `let` ein Wert zugewiesen wurde, kann dieser Wert nicht mehr geändert werden.

- Rust

```
fn main() {  
    let x = 5;  
    println!("Der Wert von x ist: {}", x);  
  
    // Der folgende Code würde einen Kompilierfehler verursachen:  
    // x = 6; // error[E0384]: cannot assign twice to immutable variable `x`  
    // println!("Der Wert von x ist jetzt: {}", x);  
}
```

-
- Warum diese Einschränkung? Immutabilität fördert die Sicherheit und erleichtert das Nachdenken über den Code. Wenn man weiß, dass sich ein Wert nach seiner Initialisierung nicht ändern kann, ist es einfacher, den Programmfluss zu verfolgen und insbesondere in parallelem Code Race Conditions zu vermeiden. Man kann es sich vorstellen wie eine Konstante in der Mathematik – ihr Wert ist festgelegt.
- **Mutabilität (Veränderlichkeit) mit `mut`:** Natürlich müssen wir manchmal Variablenwerte ändern können. Dafür können wir eine Variable explizit als **mutable** (veränderlich) deklarieren, indem wir das Schlüsselwort `mut` vor den Variablennamen stellen.

- Rust

```
fn main() {  
    let mut y = 10; // 'y' ist jetzt mutable  
    println!("Der anfängliche Wert von y ist: {}", y);  
  
    y = 12; // Das ist jetzt erlaubt, da 'y' mutable ist.  
    println!("Der Wert von y ist jetzt: {}", y);  
}
```

-
- Die Verwendung von `mut` signalisiert klar die Absicht, den Wert dieser Variablen später zu ändern. Rust ermutigt dazu, Variablen nur dann `mut` zu machen, wenn es wirklich notwendig ist.

● 2.4 Shadowing (Variablen überschatten)

Rust erlaubt ein Konzept namens **Shadowing**, das auf den ersten Blick ähnlich wie `mut` aussehen mag, aber konzeptionell anders ist. Mit `let` können wir eine neue Variable mit demselben Namen wie eine vorherige Variable

deklarieren. Diese neue Variable "überschattet" die vorherige.

- Rust

```
fn main() {  
    let x = 5;  
    println!("Der Wert von x ist: {}", x); // Gibt 5 aus  
  
    // Shadowing: Wir deklarieren eine *neue* Variable x,  
    // die die vorherige Variable x überschattet.  
    let x = x * 2; // Die neue Variable x hat den Wert 10.  
    println!("Der überschattete Wert von x ist: {}", x); // Gibt 10 aus  
  
    {  
        // Shadowing in einem inneren Gültigkeitsbereich (Scope)  
        let x = x + 1; // Neue Variable x = 11, überschattet die äußere x=10  
        println!("Der Wert von x im inneren Scope ist: {}", x); // Gibt 11 aus  
    } // Innerer Scope endet, die Variable x=11 existiert nicht mehr  
  
    // Zurück zum äußeren Scope, die Variable x=10 ist wieder sichtbar  
    println!("Der Wert von x nach dem inneren Scope ist: {}", x); // Gibt 10 aus  
  
    // Shadowing kann auch den Typ ändern:  
    let spaces = " "; // spaces ist vom Typ &str (String-Slice)  
    let spaces = spaces.len(); // spaces wird *neu* deklariert, ist jetzt vom Typ usize (Zahl)  
    println!("Anzahl Leerzeichen: {}", spaces); // Gibt 3 aus  
}
```

- Unterschied zwischen `mut` und Shadowing:

- `mut`: Ermöglicht die Änderung des Werts einer Variablen, aber nicht ihres Typs. Die Variable selbst bleibt dieselbe.
- Rust

```
let mut a = 5;  
// a = "fünf"; // Fehler! Kann den Typ einer mutablen Variablen nicht ändern.  
a = 6; // Ok
```

-
-
- Shadowing: Erstellt eine komplett *neue* Variable mit demselben Namen. Die alte Variable existiert weiterhin (ist aber nicht mehr zugreifbar, solange die neue sie überschattet). Die neue Variable kann einen anderen Typ haben. Da wir `let` verwenden, ist die neue Variable standardmäßig wieder immutable (es sei denn, wir verwenden `let mut`).

- Rust

```
let b = 5;  
let b = "fünf"; // Ok, die neue Variable 'b' ist vom Typ &str.
```

-
-
- Shadowing ist oft nützlich, wenn man eine Transformation auf einen Wert anwenden möchte, aber den transformierten Wert weiterhin unter demselben konzeptionellen Namen verwenden will (wie im `spaces`-Beispiel oben, wo wir von einem String zur seiner Länge wechseln).

Kapitel 3: Datentypen I - Skalare Typen

Jeder Wert in Rust hat einen bestimmten **Datentyp**, der dem Compiler mitteilt, welche Art von Daten der Wert repräsentiert und wie er im Speicher abgelegt und interpretiert werden soll. Rust ist, wie erwähnt, statisch typisiert. Datentypen lassen sich grob in zwei Kategorien einteilen: **skalare** und **zusammengesetzte** Typen. In diesem Kapitel konzentrieren wir uns auf die skalaren Typen. Ein skalarer Typ repräsentiert einen einzelnen Wert. Rust hat vier primäre skalare Typen: Ganzzahlen, Gleitkommazahlen, Boolesche Werte und Zeichen.

● 3.1 Ganzzahlen (Integers)

Ganzzahlen sind Zahlen ohne Nachkommastellen. Rust bietet verschiedene Ganzzahltypen, die sich in ihrer Größe (Anzahl der Bits im Speicher) und darin unterscheiden, ob sie vorzeichenbehaftet (signed) oder vorzeichenlos (unsigned) sind.

- **Größen:** Die Zahl im Typpnamen gibt die Anzahl der Bits an, die der Typ im Speicher belegt: 8, 16, 32, 64 oder 128 Bit.
- **Vorzeichenbehaftet (Signed):** Beginnen mit `i` (für Integer). Können negative und positive Zahlen sowie Null darstellen. Der Wertebereich für einen n-Bit signed Integer ist von $-(2^{(n-1)})$ bis $2^{(n-1)} - 1$.
 - `i8`: -128 bis 127
 - `i16`: -32,768 bis 32,767
 - `i32`: -2,147,483,648 bis 2,147,483,647 (Dies ist der Standard-Integer-Typ in Rust, wenn kein anderer angegeben wird: `let x = 10;` // x ist i32)
 - `i64`: Sehr großer Bereich

- `i128`: Extrem großer Bereich
 - **Vorzeichenlos (Unsigned)**: Beginnen mit `u` (für Unsigned). Können nur nicht-negative Zahlen (Null und positive Zahlen) darstellen. Der Wertebereich für einen n-Bit unsigned Integer ist von 0 bis $2^n - 1$.
 - `u8`: 0 bis 255 (oft für Bytes verwendet)
 - `u16`: 0 bis 65,535
 - `u32`: 0 bis 4,294,967,295
 - `u64`: Sehr großer Bereich
 - `u128`: Extrem großer Bereich
 - **Architekturabhängige Typen**: `isize` und `usize`. Ihre Größe hängt von der Architektur des Systems ab, auf dem das Programm kompiliert wird (32 Bit auf 32-Bit-Systemen, 64 Bit auf 64-Bit-Systemen). Diese Typen werden hauptsächlich zur Indizierung von Collections (wie Arrays oder Vektoren) verwendet, da sie garantiert groß genug sind, um jede Speicheradresse im Adressraum des Systems zu repräsentieren.
 - **Integer-Literale**: Sie können Zahlen auf verschiedene Arten schreiben:
 - Dezimal: `98_222` (Unterstriche `_` können zur besseren Lesbarkeit eingefügt werden)
 - Hexadezimal: `0xff` (entspricht 255 dezimal)
 - Oktal: `0o77` (entspricht 63 dezimal)
 - Binär: `0b1111_0000` (entspricht 240 dezimal)
 - Byte (nur für `u8`): `b'A'` (entspricht 65 dezimal, dem ASCII-Wert von 'A')
 - **Integer-Überlauf (Overflow)**: Was passiert, wenn das Ergebnis einer Operation den darstellbaren Bereich eines Typs überschreitet?
 - Im **Debug**-Modus (Standard mit `cargo run`, `cargo build`): Rust prüft auf Überlauf, und das Programm wird bei einem Überlauf mit `panic!` beendet (mehr zu `panic!` später). Dies hilft, Fehler früh zu finden.
 - Im **Release**-Modus (mit `cargo build --release`): Rust prüft *nicht* standardmäßig auf Überlauf. Stattdessen findet ein sogenanntes "Two's Complement Wrapping" statt. Das bedeutet, der Wert "läuft über" (z.B. wird `255u8 + 1u8` zu `0u8`). Wenn man ein anderes Verhalten wünscht (z.B. Sättigung oder explizite Überlaufprüfung), bietet die Standardbibliothek spezielle Methoden dafür an (z.B. `checked_add`, `wrapping_add`, `saturating_add`).
- **3.2 Gleitkommazahlen (Floats)**

Gleitkommazahlen (oder Fließkommazahlen) sind Zahlen mit Nachkommastellen. Rust hat zwei Gleitkommatypen, die beide dem IEEE-754-Standard entsprechen:

- `f32`: 32-Bit-Gleitkommazahl (single precision - einfache Genauigkeit).
- `f64`: 64-Bit-Gleitkommazahl (double precision - doppelte Genauigkeit). Dies ist der Standardtyp, wenn Sie eine Gleitkommazahl ohne explizite Typangabe schreiben, da moderne CPUs `f64` oft genauso schnell verarbeiten wie `f32`, aber eine höhere Genauigkeit bieten.

- Rust

```
fn main() {  
    let x = 2.0; // x hat den Typ f64 (Standard)  
    let y: f32 = 3.0; // y hat explizit den Typ f32  
  
    println!("x = {}, y = {}", x, y);  
}
```

-
- Gleitkommazahlen sind nützlich für viele Berechnungen, aber man sollte sich ihrer Eigenheiten bewusst sein (z.B. können sie aufgrund der internen binären Repräsentation nicht alle Dezimalzahlen exakt darstellen, was zu kleinen Rundungsfehlern führen kann).

● 3.3 Boolesche Werte (`bool`)

Der Boolesche Typ in Rust heißt `bool` und hat nur zwei mögliche Werte: `true` (wahr) oder `false` (falsch). Boolesche Werte werden hauptsächlich für Kontrollflussentscheidungen verwendet (z.B. in `if`-Bedingungen). Sie belegen ein Byte im Speicher.

- Rust

```
fn main() {  
    let ist_rust_toll = true;  
    let ist_programmieren_langweilig: bool = false; // Typannotation auch möglich  
  
    if ist_rust_toll {  
        println!("Ja, Rust ist toll!");  
    } else {  
        println!("Das sollte nicht passieren.");  
    }  
}
```

-
-

- **3.4 Zeichen (char)**

Der Typ `char` in Rust repräsentiert ein einzelnes **Unicode Scalar Value**. Das ist wichtig: Ein `char` ist *nicht* nur ein einzelnes Byte (wie oft in C für ASCII), sondern kann jedes Unicode-Zeichen darstellen, inklusive Akzenten, Symbolen oder Emojis. Daher belegt ein `char` in Rust immer **vier Bytes** im Speicher.

`char`-Literele werden in **einfachen Anführungszeichen** geschrieben ('').

- Rust

```
fn main() {  
    let c = 'z';  
    let z = 'z'; // Ein mathematisches Symbol  
    let herz = '♥';  
    let katzen_gesicht = '😺'; // Ein Emoji  
  
    println!("{}", c, z, herz, katzen_gesicht);  
}
```

-
- Die Verwendung von Unicode für `char` macht Rust gut geeignet für die Verarbeitung von Text in verschiedenen menschlichen Sprachen.

- **3.5 Grundlegende Operatoren**

Rust unterstützt die üblichen mathematischen und logischen Operatoren, die Sie vielleicht aus anderen Sprachen kennen.

- **Arithmetische Operatoren:**

- `+` (Addition)
- `-` (Subtraktion)
- `*` (Multiplikation)
- `/` (Division)
- `%` (Modulo / Rest der Division)

- Rust

```
fn main() {  
    // Addition  
    let summe = 5 + 10; // 15  
    // Subtraktion
```

```

let differenz = 95.5 - 4.3; // 91.2
// Multiplikation
let produkt = 4 * 30; // 120
// Division
let quotient = 56.7 / 32.2; // ca. 1.76... (Gleitkommazahlen)
let ganzzahl_quotient = 5 / 2; // Ergibt 2 (Integer-Division schneidet Nachkommastellen ab)
// Rest
let rest = 43 % 5; // Ergibt 3 (43 = 8 * 5 + 3)

println!("Summe: {}", summe);
println!("Differenz: {}", differenz);
println!("Produkt: {}", produkt);
println!("Quotient: {}", quotient);
println!("Ganzzahl-Quotient: {}", ganzzahl_quotient);
println!("Rest: {}", rest);
}

```

-
- **Wichtig:** Sie können arithmetische Operationen nur zwischen Werten des *gleichen* Typs durchführen (z.B. `i32 + i32`, `f64 * f64`). Eine Operation wie `5i32 + 2.0f64` ist nicht direkt erlaubt; Sie müssten einen der Werte explizit in den Typ des anderen umwandeln (Type Casting, z.B. `5i32 as f64 + 2.0f64`).
- **Boolesche Operatoren:** Werden meist mit `bool`-Werten verwendet.
 - `!` (Logisches NICHT / Negation)
 - `&&` (Logisches UND - kurzschlussauswertend)
 - `||` (Logisches ODER - kurzschlussauswertend)
- **Vergleichsoperatoren:** Vergleichen zwei Werte und ergeben einen `bool`-Wert (`true` oder `false`).
 - `==` (Gleichheit)
 - `!=` (Ungleichheit)
 - `<` (Kleiner als)
 - `>` (Größer als)
 - `<=` (Kleiner oder gleich)
 - `>=` (Größer oder gleich)
- Rust

```

fn main() {
  let a = 5;
  let b = 10;
  let gleich = a == b; // false
  let ungleich = a != b; // true
  let kleiner_als = a < b; // true
}

```

```
println!("a == b: {}", gleich);
println!("a != b: {}", ungleich);
println!("a < b: {}", kleiner_als);
}
```

-
-

Kapitel 4: Funktionen

Funktionen sind fundamentale Bausteine in Rust (und vielen anderen Programmiersprachen). Sie erlauben es uns, Code zu organisieren, zu benennen und wiederzuverwenden. Wir haben die `main`-Funktion bereits kennengelernt; jetzt schauen wir uns an, wie wir eigene Funktionen definieren und verwenden können.

• 4.1 Funktionen definieren und aufrufen

Wir definieren eine Funktion mit dem Schlüsselwort `fn`, gefolgt vom Funktionsnamen (Konvention: `snake_case`), einer Parameterliste in runden Klammern `()` und dem Funktionskörper in geschweiften Klammern `{}`.

- Rust

```
fn main() {
    println!("Nachricht aus main.");
    meine_funktion(); // Aufruf der Funktion 'meine_funktion'
    println!("Wieder zurück in main.");
}
```

```
// Definition der Funktion 'meine_funktion'
fn meine_funktion() {
    println!("Nachricht aus meine_funktion.");
}
```

-
- Wenn das Programm läuft, wird zuerst die `main`-Funktion ausgeführt. Der Aufruf `meine_funktion()` unterbricht die Ausführung von `main` temporär, springt in den Code von `meine_funktion`, führt diesen aus, und kehrt dann zur nächsten Zeile in `main` zurück.

• 4.2 Parameter und Argumente

Funktionen können Eingabewerte entgegennehmen, die als **Parameter** bezeichnet werden. Beim Aufruf einer Funktion übergeben wir konkrete Werte

für diese Parameter; diese Werte nennt man **Argumente**.

In der Funktionsdefinition müssen wir für jeden Parameter den Namen und den **Datentyp explizit angeben**. Dies ist eine Anforderung von Rusts statischem Typsystem und hilft, Fehler frühzeitig zu erkennen.

- Rust

```
fn main() {  
    addiere_zwei_zahlen(5, 10); // 5 und 10 sind die Argumente  
    addiere_zwei_zahlen(-3, 8);  
}
```

// Definition der Funktion mit zwei Parametern: x und y, beide vom Typ i32

```
fn addiere_zwei_zahlen(x: i32, y: i32) {  
    let summe = x + y;  
    println!("Die Summe von {} und {} ist: {}", x, y, summe);  
}
```

-
- Hier sind `x` und `y` die Parameter der Funktion `addiere_zwei_zahlen`, und ihre Typen (`i32`) müssen angegeben werden. Beim Aufruf `addiere_zwei_zahlen(5, 10)` ist `5` das Argument für den Parameter `x` und `10` das Argument für `y`.

• 4.3 Funktionen mit Rückgabewerten

Funktionen können nicht nur Aktionen ausführen (wie etwas ausgeben), sondern auch einen Wert berechnen und an den aufrufenden Code zurückgeben. Den Typ des Rückgabewerts deklarieren wir nach den Parametern mit einem Pfeil `->` gefolgt vom Typnamen.

- Rust

```
fn main() {  
    let ergebnis1 = addiere(5, 3); // Das Ergebnis (8) wird an 'ergebnis1' gebunden  
    println!("5 + 3 = {}", ergebnis1);  
  
    let ergebnis2 = subtrahiere_oder_null(10, 12);  
    println!("10 - 12 (oder Null) = {}", ergebnis2);  
}
```

// Diese Funktion nimmt zwei i32-Parameter und gibt einen i32 zurück

```
fn addiere(a: i32, b: i32) -> i32 {  
    // Der *letzte Ausdruck* im Funktionskörper wird automatisch zurückgegeben,  
    // WENN er *kein* Semikolon am Ende hat.
```



```
a + b
}
```

// Eine andere Möglichkeit ist die explizite Rückgabe mit dem 'return'-Schlüsselwort.
 // Das ist nützlich, wenn man nicht am Ende der Funktion zurückgeben möchte.

```
fn subtrahiere_oder_null(x: i32, y: i32) -> i32 {
    let differenz = x - y;
    if differenz < 0 {
        return 0; // Explizite Rückgabe von 0
    }
    // Wenn die Bedingung nicht zutrifft, wird dieser Ausdruck zurückgegeben
    differenz
}
```

-
- **Wichtig:** In Rust ist es idiomatischer (also gebräuchlicher), den letzten Ausdruck einer Funktion implizit zurückzugeben (ohne Semikolon), wenn dies logisch sinnvoll ist. Das `return`-Schlüsselwort wird eher für vorzeitige Rückgaben aus der Mitte einer Funktion verwendet.

• 4.4 Anweisungen vs. Ausdrücke (Statements vs. Expressions)

Dies ist ein wichtiges Konzept in Rust, das wir schon kurz berührt haben.

- **Anweisungen (Statements):** Führen eine Aktion aus, geben aber keinen Wert zurück. Die meisten Anweisungen enden mit einem Semikolon. `let x = 5;` ist eine Anweisung. Auch ein Funktionsaufruf wie `println!("Hallo");` ist eine Anweisung (obwohl `println!` technisch ein Makro ist, verhält es sich hier wie eine Anweisung).
- **Ausdrücke (Expressions):** Werten zu einem Ergebniswert aus. Die meisten Dinge in Rust sind Ausdrücke. Beispiele: `5`, `x + 1`, `true`, `addiere(3, 4)`. Auch Codeblöcke `{ ... }` sind Ausdrücke – sie werten zum Wert des letzten Ausdrucks im Block aus (wenn dieser kein Semikolon hat).

- Rust

```
fn main() {
    // 'let y = 6;' ist eine Anweisung. Sie bindet den Wert 6 an y,
    // aber die Zuweisung selbst gibt keinen Wert zurück.
    // Daher ist 'let x = (let y = 6);' ungültig.
    let y = 6; // Gültige Anweisung

    // Ein Block ist ein Ausdruck:
    let z = {
        let x = 3;
        x + 1 // Kein Semikolon hier! Dieser Ausdruck (4) ist der Rückgabewert des Blocks.
    }; // Das Semikolon hier beendet die 'let z = ...' Anweisung.
```

```
println!("Der Wert von y ist: {}", y); // Gibt 6 aus
println!("Der Wert von z ist: {}", z); // Gibt 4 aus

// Der Rückgabewert einer Funktion ist ebenfalls ein Ausdruck
let summe = addiere(10, 20); // addiere(10, 20) wertet zu 30 aus.
println!("Die Summe ist: {}", summe);
}

fn addiere(a: i32, b: i32) -> i32 {
    a + b // Ausdruck, der zurückgegeben wird
}
```

-
- Das Verständnis des Unterschieds ist entscheidend, besonders wenn wir zu Kontrollflusskonstrukten wie `if` kommen, die in Rust ebenfalls Ausdrücke sind.

Kapitel 5: Kontrollfluss I - Bedingungen

Programme müssen oft Entscheidungen treffen und ihren Ausführungspfad basierend auf bestimmten Bedingungen ändern. Das grundlegendste Werkzeug dafür ist der `if`-Ausdruck.

• 5.1 `if/else`-Ausdrücke

Ein `if`-Ausdruck erlaubt es Ihnen, Code abhängig davon auszuführen, ob eine Bedingung wahr (`true`) oder falsch (`false`) ist.

- Rust

```
fn main() {
    let zahl = 7;

    // Die Bedingung *muss* einen bool-Wert ergeben (true oder false)
    if zahl < 5 {
        println!("Bedingung war wahr (Zahl ist kleiner als 5)");
    } else {
        println!("Bedingung war falsch (Zahl ist nicht kleiner als 5)");
    }

    let zahl2 = 4;
    if zahl2 != 0 { // != bedeutet "ungleich"
        println!("Zahl ist nicht Null.");
    }

    // Ein 'else'-Block ist optional. Wenn die Bedingung false ist und
    // kein 'else' existiert, wird der if-Block einfach übersprungen.
}
```

-
- **Wichtig:** Anders als in manchen anderen Sprachen (z.B. JavaScript oder Python) wandelt Rust nicht automatisch andere Typen (wie Zahlen) in `bool` um ("truthy"/"falsy"). Die Bedingung muss explizit `true` oder `false` ergeben. `if zahl { ... }` würde nicht kompilieren, wenn `zahl` eine Zahl ist; Sie müssten `if zahl != 0 { ... }` schreiben. Die geschweiften Klammern `{ }` um die Codeblöcke sind immer erforderlich, auch wenn der Block nur eine einzige Anweisung enthält.

- **5.2** `else if`

Wenn Sie mehrere Bedingungen nacheinander prüfen möchten, können Sie `else if` verwenden. Rust prüft die Bedingungen von oben nach unten. Sobald eine `if`- oder `else if`-Bedingung `true` ist, wird der zugehörige Block ausgeführt, und alle nachfolgenden `else if`- und `else`-Blöcke werden übersprungen.

- Rust

```
fn main() {
    let zahl = 6;

    if zahl % 4 == 0 { // Prüft, ob zahl durch 4 teilbar ist
        println!("Zahl ist durch 4 teilbar.");
    } else if zahl % 3 == 0 { // Wird nur geprüft, wenn die erste Bedingung falsch war
        println!("Zahl ist durch 3 teilbar.");
    } else if zahl % 2 == 0 { // Wird nur geprüft, wenn die ersten beiden falsch waren
        println!("Zahl ist durch 2 teilbar.");
    } else { // Wird nur ausgeführt, wenn alle vorherigen Bedingungen falsch waren
        println!("Zahl ist nicht durch 4, 3 oder 2 teilbar.");
    }
    // In diesem Fall wird "Zahl ist durch 3 teilbar." ausgegeben, und die Prüfung endet.
}
```

-
- Zu viele `else if`-Ketten können unübersichtlich werden. In solchen Fällen ist oft der `match`-Operator (den wir später kennenlernen) eine bessere Alternative.

- **5.3 Bedingungen in `let`-Anweisungen**

Da `if` in Rust ein **Ausdruck** ist, kann er einen Wert zurückgeben. Diesen Wert können wir direkt in einer `let`-Anweisung verwenden.

- Rust

```
fn main() {
    let bedingung = true;
```

```
// Der Wert, der 'zahl' zugewiesen wird, hängt von der Bedingung ab.
let zahl = if bedingung {
    5 // Wert, wenn die Bedingung wahr ist
} else {
    6 // Wert, wenn die Bedingung falsch ist
}; // Semikolon beendet die let-Anweisung

println!("Der Wert von zahl ist: {}", zahl); // Gibt 5 aus

// Wichtig: Die Typen der Werte in *allen* Blöcken (if und else)
// müssen übereinstimmen! Der folgende Code würde *nicht* kompilieren:
/*
let zahl2 = if bedingung {
    5 // i32
} else {
    "sechs" // &str (String-Slice) -> Typenkonflikt!
};
*/
}
```

- Diese Fähigkeit, `if` in `let`-Anweisungen zu verwenden, führt oft zu sehr kompaktem und ausdrucksstarkem Code, da man nicht erst eine mutable Variable deklarieren muss, um ihr dann innerhalb der `if/else`-Blöcke Werte zuzuweisen. Der Compiler stellt sicher, dass die Variable `zahl` auf jeden Fall einen gültigen Wert des richtigen Typs erhält.

Damit haben wir **Teil 1: Grundlagen und erste Schritte** unseres Inhaltsverzeichnisses detailliert durchgearbeitet. Sie haben nun ein grundlegendes Verständnis von Rust, seiner Philosophie, der Installation, der grundlegenden Syntax, Variablen, einfachen Datentypen, Funktionen und bedingten Anweisungen.

Dies bildet eine solide Basis für die nächsten, sehr Rust-spezifischen Konzepte, die in Teil 2 behandelt werden, insbesondere das Ownership-System.

Teil 2: Ownership und zusammengesetzte Datentypen

Sehr gut, fahren wir also fort mit **Teil 2: Ownership und zusammengesetzte Datentypen**. Dieser Teil ist von zentraler Bedeutung, da er das **Ownership-Konzept** einführt – das wohl innovativste und prägendste Merkmal von Rust. Es ist der Mechanismus, der Rust seine Speicher- und Threadsicherheit verleiht, ohne die Notwendigkeit eines Garbage Collectors. Wir werden auch untersuchen, wie Rust Daten strukturiert, die aus mehreren Werten bestehen. Seien Sie bereit, tief einzutauchen, denn diese Konzepte sind grundlegend für das Verständnis von idiomatischem Rust-Code.

Teil 2: Ownership und zusammengesetzte Datentypen

In diesem Abschnitt verlassen wir die grundlegendsten Syntaxelemente und wenden uns komplexeren, aber essentiellen Konzepten zu. Wir beginnen mit dem Herzstück von Rusts Sicherheitsgarantien – Ownership – und behandeln dann verschiedene Möglichkeiten, Daten zu gruppieren.

Kapitel 6: Das Ownership-Konzept

Ownership (Besitz) ist Rusts Ansatz zur Speicherverwaltung. Jede Programmiersprache muss verwalten, wie ihr Code den Speicher des Computers nutzt. Einige Sprachen verwenden eine Garbage Collection (GC), die regelmäßig nach nicht mehr benötigtem Speicher sucht und ihn freigibt (z.B. Java, C#, Go, Python). Andere Sprachen erfordern vom Programmierer, Speicher explizit anzufordern und wieder freizugeben (z.B. C, C++). Rust wählt einen dritten Weg: Speicher wird durch ein System von Ownership-Regeln verwaltet, deren Einhaltung der Compiler zur Kompilierzeit überprüft. Wenn der Code kompiliert, gibt es keine Speicherfehler der Art, die Ownership verhindern soll (wie Null-Pointer-Dereferenzierungen, doppelte Speicherfreigaben oder Pufferüberläufe unter bestimmten Bedingungen) und auch keine Laufzeitkosten durch einen Garbage Collector.

- **6.1 Was ist Ownership? (Stack vs. Heap kurz erklärt)**

Um Ownership zu verstehen, müssen wir kurz betrachten, wie ein Programm Speicher verwendet. Zwei wichtige Bereiche des Speichers sind der **Stack** und der **Heap**.

- **Der Stack (Stapelspeicher):**

- Funktioniert nach dem LIFO-Prinzip (Last-In, First-Out). Wenn Sie eine Funktion aufrufen, werden ihre lokalen Variablen und einige Verwaltungsinformationen oben auf den Stack gelegt (ein "Stack Frame"). Wenn die Funktion endet, wird dieser Frame wieder vom Stack entfernt.
- Daten auf dem Stack müssen eine **bekannte, feste Größe** zur Kompilierzeit haben. Alle unsere bisherigen skalaren Typen (`i32`, `f64`, `bool`, `char`) können auf dem Stack leben.
- Die Allokation (Speicherbelegung) und Deallokation (Freigabe) auf dem Stack ist **sehr schnell**, da es nur darum geht, einen Zeiger auf das aktuelle Stack-Ende zu verschieben.
- **Analogie:** Stellen Sie sich einen Stapel Teller vor. Sie können nur Teller oben drauflegen und nur den obersten Teller wieder wegnehmen. Das Hinzufügen und Entfernen ist sehr schnell.
- **Der Heap (Halde):**
 - Wenn Sie zur Kompilierzeit Daten haben, deren Größe unbekannt ist oder sich zur Laufzeit ändern kann, oder wenn Sie möchten, dass Daten länger leben als die Funktion, die sie erstellt hat, müssen Sie Speicher auf dem Heap allokalieren.
 - Die Allokation auf dem Heap ist **langsamer** als auf dem Stack. Das Betriebssystem muss eine ausreichend große, freie Speicherstelle finden, sie als belegt markieren und einen **Zeiger** (eine Speicheradresse) auf diese Stelle zurückgeben. Dieser Zeiger wird dann typischerweise auf dem Stack gespeichert.
 - Die Herausforderung beim Heap ist die **Freigabe** des Speichers. Wann weiß das System, dass der Speicher nicht mehr benötigt wird?
 - **Manuelle Verwaltung (C/C++):** Der Programmierer muss explizit `free` oder `delete` aufrufen. Fehler hierbei (zu früh, zu spät, doppelt freigeben) sind häufige Quellen für Bugs und Sicherheitslücken.
 - **Garbage Collection (Java/Go/etc.):** Ein Hintergrundprozess (der GC) sucht periodisch nach nicht mehr referenziertem Speicher und gibt ihn frei. Das ist bequem, kann aber zu unvorhersehbaren Leistungseinbußen führen (GC-Pausen).
 - **Ownership (Rust):** Rust verwendet Ownership-Regeln, um die Lebensdauer von Heap-Daten zu verwalten und den Speicher automatisch freizugeben, sobald er nicht mehr benötigt wird (wenn der "Owner" den Gültigkeitsbereich verlässt), ohne Laufzeit-Overhead eines GC.

- **Analogie:** Stellen Sie sich vor, Sie gehen in ein Restaurant (Heap). Sie fragen nach einem Tisch (allokieren Speicher). Der Kellner sucht einen freien Tisch und gibt Ihnen die Tischnummer (den Zeiger). Wenn Sie gehen, müssen Sie dem Kellner Bescheid sagen (Speicher freigeben), damit der Tisch für andere Gäste verfügbar wird. Ownership ist Rusts System, um sicherzustellen, dass Sie *genau einmal* Bescheid sagen, wenn Sie den Tisch nicht mehr brauchen.
- **Ownership ist also primär ein System zur Verwaltung von Heap-Speicher.** Es legt fest, welcher Teil des Codes für welchen Speicherbereich auf dem Heap verantwortlich ist und wann dieser Speicher sicher freigegeben werden kann.
- **6.2 Die Ownership-Regeln**

Der Rust-Compiler erzwingt zur Kompilierzeit drei Kernregeln für Ownership:

- **Jeder Wert in Rust hat eine Variable, die sein "Owner" (Besitzer) ist.**
- Rust

```
{ // s ist hier noch nicht gültig
  let s = String::from("Hallo"); // s ist ab hier gültig und der Owner von "Hallo"
  // ... Aktionen mit s ...
} // Dieser Gültigkeitsbereich (Scope) ist nun zu Ende.
// s ist nicht mehr gültig. Rust ruft automatisch `drop` für s auf.
```

Hier ist die Variable `s` der Owner des `String`-Wertes. `String` ist ein Typ, der Textdaten auf dem Heap speichert (im Gegensatz zu String-Literalen `&str`, die wir später genauer betrachten).

2. **Es kann zu jedem Zeitpunkt immer nur *einen* Owner geben.**

Wenn wir versuchen, einen Wert einem anderen zuzuweisen, dessen Typ keine spezielle `Copy`-Eigenschaft hat (dazu gleich mehr), wird der Besitz **verschoben** (moved).

```
``rust
let s1 = String::from("Besitz");
let s2 = s1; // Besitz wird von s1 auf s2 verschoben (move)

// Nach dem Move ist s1 nicht mehr gültig!
// Der folgende Code würde einen Kompilierfehler verursachen:
// println!("{}", s1); // error[E0382]: borrow of moved value: `s1`
println!("{}", s2); // s2 ist jetzt der Owner.
```

...

Dies verhindert das "double free"-Problem: Wenn sowohl `s1` als auch `s2` versuchen würden, denselben Speicher freizugeben, wenn sie den Gültigkeitsbereich verlassen, wäre das ein Fehler. Durch den Move ist nur noch `s2` für die Freigabe verantwortlich.

3. ****Wenn der Owner aus dem Gültigkeitsbereich (Scope) geht, wird der Wert "gedroppt" (dropped).****

Das bedeutet, der zugehörige Speicher (insbesondere der Heap-Speicher) wird automatisch freigegeben. Rust fügt den Code zur Freigabe (den Aufruf einer speziellen `drop`-Funktion) am Ende des Gültigkeitsbereichs des Owners ein.

```
```rust
{
 let s_inner = String::from("Innerer Wert"); // Owner ist s_inner
 println!("{}", s_inner);
} // s_inner geht hier aus dem Scope, `drop` wird aufgerufen, Speicher freigegeben.
// println!("{}", s_inner); // Fehler! s_inner existiert hier nicht mehr.
```
```

-

-

- **6.3 Move-Semantik**

Wie in Regel 2 gesehen, wird bei Zuweisungen von Heap-basierten Daten (wie `String`, `Vec<T>`, `Box<T>`) der Besitz standardmäßig verschoben (Move). Die ursprüngliche Variable wird ungültig.

Warum ist das wichtig? Stellen Sie sich vor, `String` würde einfach den Zeiger auf die Heap-Daten kopieren (eine sogenannte "shallow copy"). Dann würden `s1` und `s2` auf denselben Speicher zeigen. Wenn `s1` aus dem Scope geht, würde der Speicher freigegeben. Wenn dann `s2` aus dem Scope geht, würde versucht, bereits freigegebenen Speicher *nochmal* freizugeben – ein schwerwiegender Fehler (double free). Rust verhindert dies durch das Move-Konzept: Nur der neue Owner (`s2`) ist gültig und verantwortlich für die Freigabe.

Analogie: Sie haben ein physisches Buch (`String`-Daten auf dem Heap). Sie geben es einer anderen Person (`let s2 = s1;`). Jetzt hat die andere Person das Buch (Ownership), Sie haben es nicht mehr. Sie können nicht mehr darin lesen (`s1` ist ungültig).

- **6.4 Clone-Semantik (`.clone()`)**

Was ist, wenn wir wirklich eine unabhängige Kopie der Heap-Daten erstellen wollen, sodass sowohl die alte als auch die neue Variable gültige Besitzer ihrer *eigenen* Daten sind? Dafür gibt es die `Clone`-Trait und die

`.clone()`-Methode. Viele Typen, einschließlich `String`, implementieren `Clone`.

- Rust

```
let s1 = String::from("Hallo");
```

```
let s2 = s1.clone(); // Erstellt eine tiefe Kopie (deep copy) der Heap-Daten.
```

```
println!("s1 = {}, s2 = {}", s1, s2); // Beides ist gültig!
```

```
// s1 und s2 zeigen jetzt auf *unterschiedliche* Speicherbereiche auf dem Heap,
```

```
// die aber den gleichen Inhalt ("Hallo") haben.
```

```
// Beide werden unabhängig voneinander gedroppt, wenn sie den Scope verlassen.
```

-
- Das Klonen kann potenziell aufwändig sein (es muss neuer Speicher allokiert und Daten kopiert werden), daher macht Rust es explizit (`.clone()`) und nicht zum Standardverhalten für Heap-Daten.

- **6.5 Copy-Semantik (für Stack-Typen)**

Die Move-Semantik gilt nicht für alle Typen. Typen, deren Daten vollständig auf dem **Stack** gespeichert werden können und eine **bekannte, feste Größe** haben, können die `Copy`-Trait implementieren. Dazu gehören alle unsere skalaren Typen (`i32`, `f64`, `bool`, `char`) sowie Tupel oder Arrays, die *nur* `Copy`-Typen enthalten.

Wenn ein Typ `Copy` implementiert, wird bei einer Zuweisung keine Verschiebung (Move) durchgeführt, sondern eine einfache, bitweise Kopie der Daten auf dem Stack erstellt. Die ursprüngliche Variable bleibt weiterhin gültig und benutzbar.

- Rust

```
let x = 5; // x ist i32, lebt auf dem Stack, implementiert Copy
```

```
let y = x; // Eine Kopie von x wird erstellt und y zugewiesen.
```

```
println!("x = {}, y = {}", x, y); // Beides ist gültig! x wurde nicht verschoben.
```

-
- Dies ist effizient, da das Kopieren von Daten auf dem Stack sehr schnell ist. Daher gibt es keinen Grund, die ursprüngliche Variable `x` ungültig zu machen.

Zusammenfassend:

- Typen, die Ressourcen auf dem Heap verwalten (wie `String`, `Vec<T>`) oder andere nicht-`Copy`-Typen enthalten: **Move**-Semantik bei Zuweisung/Funktionsübergabe. Explizites `.clone()` für tiefe Kopien.
- Typen, die vollständig auf dem Stack liegen und `Copy` implementieren (wie `i32`, `bool`, Tupel aus `Copy`-Typen): **Copy**-Semantik bei Zuweisung/Funktionsübergabe. Original bleibt gültig.
- **6.6 Ownership und Funktionen**

Die Ownership-Regeln gelten auch, wenn Werte an Funktionen übergeben oder von Funktionen zurückgegeben werden.

- **Übergabe an Funktion:**
 - Das Übergeben eines Wertes an eine Funktion entspricht einer Zuweisung.
 - Für `Copy`-Typen wird eine Kopie übergeben, der ursprüngliche Wert bleibt gültig.
 - Für Move-Typen (`String`, etc.) wird der Besitz an den Funktionsparameter **verschoben**. Die Variable im aufrufenden Code wird ungültig.
- Rust

```
fn main() {
    let s = String::from("übergeben"); // s ist Owner
    nimmt_ownership(s); // Ownership von s wird an die Funktion verschoben

    // println!("{}", s); // Fehler! s ist hier nicht mehr gültig.

    let x = 5; // x ist i32 (Copy)
    macht_kopie(x); // Kopie von x wird an die Funktion übergeben
    println!("x ist immer noch gültig: {}", x); // x ist hier noch gültig.
}
```

```
fn nimmt_ownership(some_string: String) { // some_string wird Owner
    println!("{}", some_string);
} // some_string geht aus dem Scope, drop wird aufgerufen. Speicher wird freigegeben.
```

```
fn macht_kopie(some_integer: i32) { // some_integer erhält eine Kopie
    println!("{}", some_integer);
} // some_integer geht aus dem Scope, nichts Besonderes passiert.
```

-
-
- **Rückgabe von Funktion:**

- Funktionen können Werte zurückgeben und damit auch Ownership übertragen.

- Rust

```
fn main() {
    let s1 = gibt_ownership(); // s1 wird Owner des zurückgegebenen Strings
    println!("{}", s1);

    let s2 = String::from("original");
    let s3 = nimmt_und_gibt_ownership_zurück(s2); // s2 wird verschoben, s3 wird neuer Owner
    // println!("{}", s2); // Fehler! s2 wurde verschoben.
    println!("{}", s3);
}

fn gibt_ownership() -> String {
    let some_string = String::from("deins");
    some_string // Gibt Ownership des Strings zurück
}

fn nimmt_und_gibt_ownership_zurück(a_string: String) -> String {
    println!("Habe kurz Besitz von: {}", a_string);
    a_string // Gibt Ownership zurück
}
```

- Dieses ständige Verschieben von Ownership kann umständlich werden, wenn eine Funktion Daten nur *lesen* oder *modifizieren*, aber nicht den Besitz übernehmen soll. Man müsste den Wert in die Funktion verschieben und dann wieder zurückgeben (wie in `nimmt_und_gibt_ownership_zurück`). Das führt uns direkt zum nächsten Konzept: Referenzen und Borrowing.

Kapitel 7: Referenzen und Borrowing (Leihen)

Das Problem: Wie können wir einer Funktion Zugriff auf Daten gewähren, ohne ihr die Ownership zu übertragen? Die Lösung sind **Referenzen**. Eine Referenz ist wie eine Adresse, die es Ihnen ermöglicht, auf Daten zuzugreifen, die einem anderen Owner gehören, ohne selbst der Owner zu werden. Diesen Vorgang des Erstellens von Referenzen nennen wir **Borrowing** (Leihen oder Borgen).

• 7.1 Das Problem mit Ownership und Datenweitergabe

Stellen Sie sich vor, wir wollen die Länge eines `String` berechnen, ohne dass die Funktion den `String` konsumiert (seine Ownership übernimmt). Mit reinem

Ownership müssten wir so etwas tun:

- Rust

```
fn main() {  
    let s1 = String::from("Hallo");  
    let (s2, len) = berechne_laenge_umstaendlich(s1); // Ownership geht an Funktion, kommt zurück  
    println!("Die Länge von '{}' ist {}.", s2, len);  
}
```

```
fn berechne_laenge_umstaendlich(s: String) -> (String, usize) {  
    let length = s.len(); // len() gibt die Länge zurück  
    (s, length) // Ownership des Strings und die Länge zurückgeben  
}
```

-
- Das ist unhandlich, besonders wenn wir mehrere Werte oder den Wert in mehreren Funktionen verwenden wollen.

- **7.2 Unveränderliche Referenzen (&)**

Statt den Wert selbst zu übergeben, können wir eine **Referenz** darauf übergeben. Wir erstellen eine Referenz mit dem **&**-Operator und geben den Typ mit **&Typ** an.

- Rust

```
fn main() {  
    let s1 = String::from("Hallo");  
  
    // Wir übergeben eine Referenz auf s1 (&s1)  
    // Der Typ des Parameters ist &String  
    let len = berechne_laenge_mit_referenz(&s1);  
  
    println!("Die Länge von '{}' ist {}.", s1, len); // s1 ist immer noch gültig!  
}
```

```
// Die Funktion nimmt eine *unveränderliche Referenz* auf einen String  
fn berechne_laenge_mit_referenz(s: &String) -> usize {  
    // Wir können s lesen, aber nicht verändern!  
    // s.push_str(" Welt"); // Fehler! s ist eine unveränderliche Referenz.  
    s.len()  
} // s geht hier aus dem Scope, aber da es keine Ownership hat,  
// wird der String, auf den es zeigt, *nicht* gedroppt.
```

-

- Hier leiht sich (`borrow`s) die Funktion `berechne_laenge_mit_referenz` den String `s1` nur aus. Sie erhält eine Referenz (`&s1`), die ihr erlaubt, den Wert zu lesen (`s.len()`), aber nicht zu verändern. Nach dem Funktionsaufruf ist `s1` in `main` immer noch der gültige Owner.

Analogie: Sie leihen einem Freund ein Buch (`&String`), aber er darf es nur lesen, nicht hineinschreiben. Sie bleiben der Besitzer des Buches (`s1` bleibt Owner).

- **7.3 Veränderliche Referenzen (`&mut`)**

Was, wenn wir einer Funktion erlauben wollen, den Wert, den sie sich leiht, auch zu **modifizieren**? Dafür gibt es **mutable Referenzen**, erstellt mit `&mut` und mit dem Typ `&mut Typ`.

- Rust

```
fn main() {
    // Wichtig: Die ursprüngliche Variable muss 'mut' sein,
    // um eine mutable Referenz darauf erstellen zu können!
    let mut s = String::from("Hallo");

    // Wir übergeben eine *mutable* Referenz auf s (&mut s)
    // Der Typ des Parameters ist &mut String
    aendere_string(&mut s);

    println!("{}", s); // Gibt "Hallo, Welt" aus. s wurde verändert!
}

// Die Funktion nimmt eine *veränderliche Referenz* auf einen String
fn aendere_string(some_string: &mut String) {
    // Wir können den String jetzt ändern!
    some_string.push_str(", Welt");
}
```

-
- Hier leiht sich die Funktion `aendere_string` den String `s` veränderbar aus. Sie kann Methoden aufrufen, die den String modifizieren (wie `push_str`).

Analogie: Sie leihen einem Freund ein Buch (`&mut String`) und erlauben ihm explizit, Notizen hineinzuschreiben. Sie sind immer noch der Besitzer, aber der Inhalt des Buches kann sich ändern.

- **7.4 Die Borrowing-Regeln (entscheidend!)**

Referenzen sind mächtig, aber um Rusts Sicherheitsgarantien (insbesondere

die Verhinderung von Data Races) aufrechtzuerhalten, erzwingt der Compiler zur Kompilierzeit sehr strikte Regeln für das Borrowing. Diese Regeln sind der Kern des **Borrow Checkers**:

- **Regel 1 (Immutable Borrows):** Zu jedem Zeitpunkt können Sie **beliebig viele unveränderliche Referenzen (&T)** auf einen bestimmten Wert haben.

- Rust

```
let s = String::from("viele leser");
let r1 = &s;
let r2 = &s;
println!("{}", r1, r2); // Erlaubt: Mehrere Leser gleichzeitig sind ok.
```

-
-
- **Regel 2 (Mutable Borrows):** Zu jedem Zeitpunkt können Sie **nur eine einzige veränderliche Referenz (&mut T)** auf einen bestimmten Wert haben.

- Rust

```
let mut s = String::from("nur ein schreiber");
let r1 = &mut s;
// let r2 = &mut s; // Fehler! Kann nicht zweimal mutable leihen.
// println!("{}", r1, r2);
println!("{}", r1); // Erlaubt: Nur eine mutable Referenz ist ok.
```

-
-
- **Regel 3 (Keine Mischung):** Wenn Sie eine veränderliche Referenz (&mut T) haben, können Sie **keine weiteren Referenzen (weder &T noch &mut T)** auf diesen Wert haben. Kurz gesagt: Entweder *ein* Schreiber *oder* beliebig viele Leser, aber niemals beides gleichzeitig.

- Rust

```
let mut s = String::from("mischen?");
let r1 = &s; // Ok: immutable borrow
let r2 = &s; // Ok: noch ein immutable borrow
// let r3 = &mut s; // Fehler! Kann nicht mutable leihen, während immutable borrows existieren.
```

```
// println!("{}", {}, {}, r1, r2, r3);

// --- anderes Beispiel ---
let mut s2 = String::from("mischen andersrum?");
let r_mut = &mut s2; // Ok: mutable borrow
// let r_immut = &s2; // Fehler! Kann nicht immutable leihen, während ein mutable borrow
// existiert.
// println!("{}", {}, {}, r_mut, r_immut);
println!("{}", r_mut); // Nachdem r_mut nicht mehr verwendet wird (sein Scope endet hier quasi),
// könnten neue Referenzen erstellt werden.
```

-
-
- **Warum diese Regeln?** Sie verhindern **Data Races** zur Kompilierzeit. Ein Data Race tritt auf, wenn:
 - Zwei oder mehr Zeiger/Referenzen gleichzeitig auf dieselben Daten zugreifen.
 - Mindestens einer der Zugriffe ein schreibender Zugriff ist.
 - Es keinen Mechanismus zur Synchronisation der Zugriffe gibt. Data Races führen zu undefiniertem Verhalten und sind schwer zu debuggen. Rusts Borrowing-Regeln machen sie unmöglich: Regel 3 verhindert gleichzeitiges Lesen und Schreiben, und Regel 2 verhindert gleichzeitiges Schreiben und Schreiben.
- **7.5 Dangling References (ungültige Referenzen vermeiden)**

Ein weiteres häufiges Problem in Sprachen wie C/C++ sind **Dangling Pointers/References**: Ein Zeiger, der auf einen Speicherbereich zeigt, der bereits freigegeben wurde oder dessen Inhalt nicht mehr gültig ist. Der Versuch, über einen solchen Zeiger zuzugreifen, führt oft zu Abstürzen oder unerwartetem Verhalten.

Rust **verhindert** Dangling References durch den Borrow Checker und das Konzept der **Lifetimes** (Lebensdauern), das wir in Kapitel 22 genauer betrachten. Die Grundidee ist: Der Compiler stellt sicher, dass eine Referenz niemals länger gültig ist als die Daten, auf die sie zeigt.

Betrachten wir ein Beispiel, das in Rust *nicht* kompiliert, weil es eine Dangling Reference erzeugen würde:

- Rust

```
/* // Dieser Code kompiliert nicht!
fn main() {
    let reference_to_nothing = dangle();
```

```
}
```

```
fn dangle() -> &String { // dangle gibt eine Referenz auf einen String zurück
    let s = String::from("hello"); // s wird innerhalb von dangle erstellt

    &s // Wir versuchen, eine Referenz auf s zurückzugeben
} // Hier endet der Scope von dangle. s wird gedroppt, sein Speicher freigegeben.
// Die zurückgegebene Referenz würde nun auf ungültigen Speicher zeigen!
*/
```

-
- Der Rust-Compiler erkennt dieses Problem:
error[E0106]: missing lifetime specifier ... this function's return type contains a borrowed value, but is owned by the current function
Er sagt uns im Grunde, dass wir versuchen, eine Referenz auf etwas zurückzugeben, das nur innerhalb der Funktion existiert und am Ende der Funktion zerstört wird. Das ist nicht erlaubt. Die Lösung wäre hier, die String-Ownership selbst zurückzugeben: `fn no_dangle() -> String { ... }`.

Durch die Kombination von Ownership, Borrowing-Regeln und Lifetime-Prüfungen garantiert Rust zur Kompilierzeit, dass Referenzen immer auf gültige Daten zeigen.

Kapitel 8: Slices

Slices sind eine weitere Art von Referenz in Rust, die keine Ownership hat. Sie erlauben es Ihnen, auf eine **zusammenhängende Sequenz von Elementen** in einer Sammlung (wie einem String, Array oder Vektor) zu verweisen, anstatt auf die gesamte Sammlung.

- **8.1 Was sind Slices?**

Ein Slice ist im Grunde ein "Blick" oder eine "Scheibe" in einen Teil einer anderen Datenstruktur. Sie bestehen intern typischerweise aus einem Zeiger auf das erste Element der Sequenz und einer Längenangabe. Da sie nur Referenzen sind, folgen sie den Borrowing-Regeln (sie können mutable oder immutable sein).

- **8.2 String-Slices (&str)**

Der häufigste Slice-Typ ist der **String-Slice**, bezeichnet als `&str`. Er ist eine unveränderliche Referenz auf einen Teil eines `String`-Wertes (oder eines anderen String-Slices).

- Rust

```
fn main() {
    let s = String::from("Hallo Welt"); // Eine String-Variable (owned, auf dem Heap)

    // Erstellen von String-Slices mit Bereichssyntax [start..end]
    // Indizes beziehen sich auf Bytes!
    let hallo = &s[0..5]; // Slice, der auf die Bytes für "Hallo" zeigt
    let welt = &s[6..11]; // Slice, der auf die Bytes für "Welt" zeigt

    // [..end] ist äquivalent zu [0..end]
    let hallo_auch = &s[..5];
    // [start..] ist äquivalent zu [start..length]
    let welt_auch = &s[6..];
    // [..] ist äquivalent zu [0..length] - ein Slice des gesamten Strings
    let ganze_s = &s[..];

    println!("{}", hallo, welt); // Gibt "Hallo, Welt" aus
    println!("{}", hallo_auch, welt_auch);
    println!("{}", ganze_s);

    // String-Literale sind bereits vom Typ &str!
    let s_literal: &str = "Ich bin ein String-Literal.";
    println!("{}", s_literal);

    // Funktion, die einen String-Slice akzeptiert
    let wort = erstes_wort(&s);
    println!("Das erste Wort ist: '{}'", wort);

    // Wichtig: Weil wir einen immutable Slice 'wort' haben, können wir s nicht mehr mutable
    // leihen!
    // s.clear(); // Fehler! cannot borrow `s` as mutable because it is also borrowed as immutable
    // println!("Das erste Wort ist immer noch: '{}'", wort); // 'wort' wäre sonst dangling!
}

// Diese Funktion nimmt einen String-Slice und gibt einen String-Slice zurück.
// Sie funktioniert sowohl mit &String als auch mit &str!
fn erstes_wort(s: &str) -> &str {
    let bytes = s.as_bytes(); // Wandelt den String in ein Byte-Array um

    for (i, &item) in bytes.iter().enumerate() {
        if item == b' ' { // Prüft auf Leerzeichen-Byte
            return &s[0..i]; // Gibt einen Slice vom Anfang bis zum Leerzeichen zurück
        }
    }
    &s[..] // Wenn kein Leerzeichen gefunden wird, ist der ganze String das erste Wort
}
```

-

- String-Slices (`&str`) sind sehr praktisch, da sie es ermöglichen, mit Teilen von Strings zu arbeiten, ohne neue `String`-Objekte (und damit Heap-Allokationen) erstellen zu müssen. Die Funktion `erstes_wort` ist flexibel, da sie `&str` als Parameter akzeptiert und somit sowohl mit `String`-Referenzen (`&s`) als auch mit `String`-Literalen oder anderen Slices aufgerufen werden kann.

Achtung bei UTF-8: Die Indizes `[start..end]` bei `String`-Slices beziehen sich auf **Bytes**, nicht auf Zeichen. Da `Rust` Strings als UTF-8 kodiert, kann ein einzelnes Zeichen (wie 'ü' oder '😊') aus mehreren Bytes bestehen. Ein Slice mitten in einem Multibyte-Zeichen würde zu einem Laufzeitfehler (`panic`) führen. Das Iterieren über Zeichen ist oft sicherer als byte-basiertes Slicing, wenn man mit Unicode arbeitet.

- **8.3 Andere Slices (z.B. von Arrays)**

Das Slice-Konzept funktioniert nicht nur für Strings, sondern auch für andere zusammenhängende Sammlungen wie Arrays oder Vektoren. Der Typ eines Slices auf ein Array von `i32`-Werten ist `&[i32]`.

- Rust

```
fn main() {
    let a: [i32; 5] = [10, 20, 30, 40, 50]; // Ein Array

    // Erstellen eines Slices, der die Elemente mit Index 1 und 2 enthält (20, 30)
    let slice: &[i32] = &a[1..3];

    println!("Array: {:?}", a);
    println!("Slice: {:?}", slice); // Gibt "[20, 30]" aus

    // Mutable Slices sind auch möglich, wenn das Original mutable ist
    let mut b = [1, 2, 3, 4, 5];
    let mutable_slice: &mut [i32] = &mut b[2..4]; // Elemente 3 und 4
    mutable_slice[0] = 33; // Ändert das Element an Index 2 im Original-Array b
    mutable_slice[1] = 44; // Ändert das Element an Index 3 im Original-Array b

    println!("Geändertes Array b: {:?}", b); // Gibt "[1, 2, 33, 44, 5]" aus
    println!("Mutable Slice: {:?}", mutable_slice); // Gibt "[33, 44]" aus
}
```

- Slices bieten eine sichere und effiziente Möglichkeit, mit Teilen von Arrays oder Vektoren zu arbeiten, ohne Daten kopieren zu müssen. Sie werden oft in Funktionssignaturen verwendet, um Funktionen zu schreiben, die auf Teilen einer Sammlung operieren können.

Kapitel 9: Datentypen II - Zusammengesetzte Typen

Nachdem wir die skalaren Typen (einzelne Werte) und das Ownership-System behandelt haben, wenden wir uns nun den **zusammengesetzten Typen** zu, die mehrere Werte zu einer Einheit gruppieren. Rust bietet hauptsächlich zwei primitive zusammengesetzte Typen: Tupel und Arrays. Structs, die wir im nächsten Kapitel behandeln, sind eine mächtigere Form.

- **9.1 Tupel**

Ein Tupel ist eine **geordnete Sammlung von Werten unterschiedlicher Typen** mit einer **festen Größe**. Einmal deklariert, kann ein Tupel weder wachsen noch schrumpfen.

- **Definition und Instanziierung:** Tupel werden durch eine komma-separierte Liste von Werten in runden Klammern `()` erstellt. Der Typ wird durch die Typen der enthaltenen Elemente in Klammern definiert.

- Rust

```
fn main() {  
    // Tupel mit i32, f64, u8  
    let tup: (i32, f64, u8) = (500, 6.4, 1);  
  
    // Typinferenz funktioniert auch  
    let tup2 = (-10, true, 'c'); // Typ ist (i32, bool, char)  
}
```

-
-
- **Zugriff auf Elemente:** Man kann auf die Elemente eines Tupels über einen Punkt `.` gefolgt vom **Index** (beginnend bei 0) zugreifen.

- Rust

```
let tup: (i32, f64, u8) = (500, 6.4, 1);  
let five_hundred = tup.0;  
let six_point_four = tup.1;  
let one = tup.2;  
println!("Werte: {}, {}, {}", five_hundred, six_point_four, one);
```

-

-
- **Destrukturierung:** Eine sehr gebräuchliche Methode, um an die Werte in einem Tupel zu gelangen, ist die **Destrukturierung** mit `let`.

- Rust

```
let tup: (i32, f64, u8) = (500, 6.4, 1);
let (x, y, z) = tup; // Destrukturiert tup in die Variablen x, y, z
```

```
println!("y hat den Wert: {}", y); // Gibt 6.4 aus
```

-
- Dies ist oft lesbarer als der Zugriff über Indizes.
- **Das Unit-Tupel ():** Ein Tupel ohne Elemente, geschrieben als `()`, wird als **Unit-Typ** bezeichnet. Es hat nur einen möglichen Wert, ebenfalls `()`. Funktionen, die "nichts" zurückgeben (wie unsere bisherigen `main`- oder `println`-Beispiele), geben implizit den Unit-Typ zurück.

- Tupel sind nützlich, wenn man eine kleine, feste Anzahl von Werten unterschiedlicher Typen gruppieren möchte, z.B. um mehrere Werte aus einer Funktion zurückzugeben (obwohl Structs mit benannten Feldern oft klarer sind).

● 9.2 Arrays

Ein Array ist eine Sammlung von Werten des **gleichen Typs** mit einer **festen Größe**. Wie Tupel können Arrays nach ihrer Erstellung nicht mehr wachsen oder schrumpfen.

- **Definition und Instanziierung:** Arrays werden mit eckigen Klammern `[]` geschrieben. Der Typ wird als `[Typ; Größe]` angegeben.

- Rust

```
fn main() {
    // Array mit 5 i32-Werten
    let a: [i32; 5] = [1, 2, 3, 4, 5];

    // Typinferenz funktioniert auch
    let b = [6.0, 7.0, 8.0]; // Typ ist [f64; 3]

    // Initialisierung mit demselben Wert: [Wert; Größe]
```

```
let c = [3; 5]; // Entspricht [3, 3, 3, 3, 3]
println!("c: {:?}", c);
}
```

-
-
- **Speicherort:** Arrays werden in der Regel als ein zusammenhängender Speicherblock auf dem **Stack** abgelegt (im Gegensatz zu dynamischen Sammlungen wie Vektoren, die wir später sehen und die ihre Daten auf dem Heap speichern).
- **Zugriff auf Elemente:** Der Zugriff erfolgt über **Indizes** in eckigen Klammern, beginnend bei 0.
- Rust

```
let a = [10, 20, 30, 40, 50];
let first = a[0]; // 10
let second = a[1]; // 20
println!("Erstes Element: {}, Zweites Element: {}", first, second);
```

-
-
- **Bounds Checking (Grenzenprüfung):** Rust prüft zur Laufzeit, ob der verwendete Index innerhalb der gültigen Grenzen des Arrays liegt. Ein Zugriff auf einen ungültigen Index (z.B. `a[10]` für ein Array der Größe 5) führt zu einem sofortigen Programmabbruch (**panic**). Dies verhindert Pufferüberläufe, eine häufige Sicherheitslücke in anderen Sprachen.
- Rust

```
/* // Dieser Code würde panic!en:
fn main() {
    let a = [1, 2, 3];
    println!("Ungültiger Zugriff: {}", a[5]); // panic! index out of bounds
}
*/
```

-
-
- Arrays sind nützlich, wenn Sie eine Sammlung mit einer festen Anzahl von Elementen benötigen, deren Größe zur Kompilierzeit bekannt ist, und wenn Sie die Garantie haben wollen, dass die Daten zusammenhängend im Speicher liegen (was gut für die Cache-Performance sein kann). Wenn Sie eine Sammlung benötigen, deren Größe sich zur Laufzeit ändern kann, ist ein

Vektor (`Vec<T>`) die geeignetere Wahl (siehe Teil 3).

Kapitel 10: Structs - Eigene Datentypen erstellen

Während Tupel und Arrays nützlich sind, reichen sie oft nicht aus, um komplexere Daten sinnvoll zu modellieren. Hier kommen **Structs** (Strukturen) ins Spiel. Structs erlauben es Ihnen, eigene, benutzerdefinierte Datentypen zu erstellen, indem Sie verschiedene Felder (Attribute) mit Namen und Typen zusammenfassen.

- **10.1 Structs definieren und instanziiieren**

Wir definieren einen Struct mit dem Schlüsselwort `struct`, gefolgt vom Namen des Structs (Konvention: `PascalCase`) und den Feldern in geschweiften Klammern `{}`. Jedes Feld hat einen Namen (Konvention: `snake_case`) und einen Typ.

- Rust

```
// Definition eines Structs 'User'
struct User {
    active: bool,
    username: String, // Beachten Sie: String (owned), nicht &str (borrowed Slice)
    email: String,
    sign_in_count: u64,
}

fn main() {
    // Instanziierung eines Structs
    // Wir müssen Werte für alle Felder angeben (Reihenfolge egal)
    let user1 = User {
        email: String::from("jemand@example.com"),
        username: String::from("Benutzername123"),
        active: true,
        sign_in_count: 1,
    };

    // Field Init Shorthand: Wenn eine Variable oder ein Funktionsparameter
    // denselben Namen wie ein Struct-Feld hat, können wir ': Feldname' weglassen.
    let email = String::from("anderer@example.com");
    let username = String::from("AndererNutzer");
    let user2 = build_user(email, username); // Verwenden einer Hilfsfunktion

    // Struct Update Syntax: Erstellt eine neue Instanz, die die meisten
    // Felder einer anderen Instanz übernimmt, aber einige ändert.
    let user3 = User {
```

```

        email: String::from("noch@einer.org"),
        // Übernimmt username, active, sign_in_count von user1
        ..user1
    };
    // Achtung: Weil user1.username (String, kein Copy-Typ) in user3
    // *verschoben* wurde, ist user1.username jetzt ungültig!
    // user1.email (auch String) wurde nicht verwendet und bleibt gültig.
    // active und sign_in_count (bool, u64 - Copy-Typen) wurden kopiert.
    // Daher ist user1 als Ganzes nicht mehr vollständig nutzbar, wenn Felder verschoben wurden.

    println!("User 2 E-Mail: {}", user2.email);
    println!("User 3 Username: {}", user3.username);
    // println!("User 1 Username: {}", user1.username); // Fehler! Wurde nach user3 verschoben.
}

// Hilfsfunktion zur Erstellung eines Users (demonstriert Field Init Shorthand)
fn build_user(email: String, username: String) -> User {
    User {
        email, // Kurzform für email: email,
        username, // Kurzform für username: username,
        active: true,
        sign_in_count: 1,
    }
}

```

• 10.2 Zugriff auf Struct-Felder

Der Zugriff auf die Felder einer Struct-Instanz erfolgt über die **Punktnotation** `(.)`.

- Rust

```

struct User {
    active: bool,
    username: String,
    email: String,
    sign_in_count: u64,
}

fn main() {
    // Instanz muss 'mut' sein, um Felder ändern zu können
    let mut user1 = User {
        email: String::from("jemand@example.com"),
        username: String::from("Benutzername123"),
        active: true,
        sign_in_count: 1,
    }
}

```

```
};

println!("Ursprüngliche E-Mail: {}", user1.email);

// Feld ändern
user1.email = String::from("neue.email@example.com");
user1.sign_in_count += 1;

println!("Neue E-Mail: {}", user1.email);
println!("Anmeldezahl: {}", user1.sign_in_count);
}
```

-
- Wenn die Struct-Instanz selbst immutable ist (`let user1 = ...`), können ihre Felder nicht geändert werden, selbst wenn sie `mut` wären (was für Felder nicht direkt möglich ist, nur für die Instanz).

• 10.3 Tuple Structs

Rust erlaubt auch **Tuple Structs**. Das sind Structs, die benannt sind, aber deren Felder keine Namen haben, ähnlich wie Tupel. Sie werden mit `struct Name(Typ1, Typ2, ...)` definiert.

- Rust

```
// Tuple Structs für Farben und Punkte
struct Color(i32, i32, i32); // RGB
struct Point(i32, i32, i32); // x, y, z

fn main() {
    let black = Color(0, 0, 0);
    let origin = Point(0, 0, 0);

    // Zugriff auf Elemente wie bei Tupeln mit Punkt und Index
    println!("Schwarz - Rotanteil: {}", black.0);
    println!("Ursprung - x-Koordinate: {}", origin.0);

    // Wichtig: Obwohl black und origin die gleiche Struktur (3x i32) haben,
    // sind sie *unterschiedliche Typen* (Color vs. Point).
    // Man kann sie nicht einfach austauschbar verwenden.
    // let p: Point = black; // Fehler! expected struct `Point`, found struct `Color`
}
```

-
- Tuple Structs sind nützlich, wenn die Namen der Felder nicht wichtig sind oder wenn man einfach nur einen neuen Typ schaffen will, der sich von einem normalen Tupel unterscheidet, um die Typsicherheit zu erhöhen (wie im `Color`

vs. `Point`-Beispiel).

- 10.4 Unit-like Structs

Es ist auch möglich, Structs ohne jegliche Felder zu definieren, sogenannte Unit-like Structs.

- Rust

```
struct AlwaysEqual;
```

```
fn main() {  
    let subject = AlwaysEqual;  
    // Nützlich, wenn man ein Verhalten (Trait) auf einem Typ implementieren  
    // möchte, der selbst keine Daten benötigt.  
}
```

-
-

- 10.5 Methoden für Structs definieren (`impl`)

Eine der Stärken von Structs ist, dass wir Verhalten direkt mit ihnen verknüpfen können, indem wir **Methoden** definieren. Methoden sind wie Funktionen, aber sie werden im Kontext einer Struct-Instanz (oder Enum/Trait) definiert und aufgerufen. Wir definieren Methoden innerhalb eines `impl`-Blocks (Implementierungsblock).

- Rust

```
struct Rectangle {  
    width: u32,  
    height: u32,  
}
```

// Implementierungsblock für den Struct Rectangle

```
impl Rectangle {  
    // Eine Methode. Der erste Parameter ist immer 'self' (oder &self, &mut self).  
    // '&self' ist eine Kurzform für 'self: &Self'. 'Self' ist ein Alias für den Typ,  
    // für den der impl-Block gilt (hier: Rectangle).  
    // Diese Methode leiht sich die Instanz unveränderlich aus (&self).  
    fn area(&self) -> u32 {  
        self.width * self.height // Greift auf Felder der Instanz zu  
    }  
}
```

// Eine Methode, die eine andere Rectangle-Instanz als Parameter nimmt

```
fn can_hold(&self, other: &Rectangle) -> bool {  
    self.width > other.width && self.height > other.height  
}
```

```

}

// Eine Methode, die die Instanz veränderlich leiht (&mut self)
fn scale(&mut self, factor: u32) {
    self.width *= factor;
    self.height *= factor;
}

// Es gäbe auch Methoden, die Ownership übernehmen (self),
// aber das ist weniger häufig.
}

fn main() {
    let mut rect1 = Rectangle { width: 30, height: 50 };
    let rect2 = Rectangle { width: 10, height: 40 };
    let rect3 = Rectangle { width: 60, height: 45 };

    // Aufruf der Methode 'area' mit Punktnotation auf der Instanz
    println!(
        "Die Fläche des Rechtecks ist {} Quadratpixel.",
        rect1.area() // Methodenaufruf
    );

    println!("Kann rect1 rect2 aufnehmen? {}", rect1.can_hold(&rect2));
    println!("Kann rect1 rect3 aufnehmen? {}", rect1.can_hold(&rect3));

    // Aufruf einer mutable Methode
    rect1.scale(2);
    println!("Skalierte Fläche: {}", rect1.area()); // Jetzt 6000
}

```

-
- Der erste Parameter einer Methode ist fast immer `self`, `&self` oder `&mut self`, was bestimmt, ob die Methode Ownership übernimmt, sich die Instanz unveränderlich leiht oder sie veränderlich leiht. Methoden werden mit der Punktnotation aufgerufen, z.B. `instanz.methode()`.

• 10.6 Assoziierte Funktionen (wie Konstruktoren)

Neben Methoden (die `self` als ersten Parameter haben) können `impl`-Blöcke auch **assozierte Funktionen** definieren. Das sind Funktionen, die mit dem Typ selbst assoziiert sind, aber nicht auf einer spezifischen Instanz operieren (sie haben keinen `self`-Parameter). Sie werden oft für Konstruktoren verwendet, um neue Instanzen des Structs zu erstellen.

Assoziierte Funktionen werden mit dem **Doppelpunktoperator** `::` auf dem Typnamen aufgerufen (z.B. `Typ::funktion()`).

- Rust

```

struct Rectangle {
    width: u32,
    height: u32,
}

impl Rectangle {
    // Eine assoziierte Funktion, oft als Konstruktor verwendet.
    // Konventionell oft 'new' genannt, aber jeder Name ist möglich.
    // Gibt eine neue Instanz des Typs 'Self' (also Rectangle) zurück.
    fn new(width: u32, height: u32) -> Self {
        Self { width, height } // Self ist Alias für Rectangle hier
    }

    // Eine weitere assoziierte Funktion, z.B. für ein Quadrat
    fn square(size: u32) -> Self {
        Self { width: size, height: size }
    }

    // 'area' ist eine Methode (hat &self)
    fn area(&self) -> u32 {
        self.width * self.height
    }
}

fn main() {
    // Aufruf der assoziierten Funktion 'new' über den Typtypen
    let rect1 = Rectangle::new(30, 50);
    // Aufruf der assoziierten Funktion 'square'
    let sq1 = Rectangle::square(20);

    println!("Fläche rect1: {}", rect1.area()); // Methodenaufruf auf Instanz
    println!("Fläche sq1: {}", sq1.area()); // Methodenaufruf auf Instanz
}

```

-
- `String::from()` ist ein Beispiel für eine assoziierte Funktion, die wir bereits verwendet haben. Sie ist mit dem Typ `String` assoziiert und erstellt eine neue `String`-Instanz.

Puh, das war ein sehr dichter und konzeptionell anspruchsvoller Teil! Wir haben das Kernkonzept von Rust – **Ownership**, **Borrowing** und **Slices** – detailliert beleuchtet. Diese Regeln sind es, die Rusts Speichersicherheit ohne Garbage Collector ermöglichen. Wir haben auch gelernt, wie wir Daten mit **Tupeln**, **Arrays** und den flexibleren **Structs** (inklusive Methoden und assoziierten Funktionen) strukturieren können.

Diese Konzepte aus Teil 2 sind absolut fundamental. Nehmen Sie sich Zeit, sie zu verstehen und damit zu experimentieren. Die Ownership- und Borrowing-Regeln mögen anfangs restriktiv erscheinen, aber sie werden Ihnen bald in Fleisch und Blut übergehen und sind der Schlüssel zu sicherem und effizientem Rust-Code.

Teil 3: Enums, Pattern Matching und Collections

Ausgezeichnet, wir schreiten gut voran! Nun widmen wir uns **Teil 3: Enums, Pattern Matching und Collections**. Dieser Abschnitt ist besonders spannend, da er uns in einige der ausdrucksstärksten und nützlichsten Features von Rust einführt. Wir lernen, wie wir Typen definieren, die eine Auswahl aus mehreren Möglichkeiten darstellen (**Enums**), und wie wir diese mit dem mächtigen **Pattern Matching** elegant verarbeiten. Außerdem tauchen wir in die Welt der **Collections** ein – Datenstrukturen wie Vektoren, Strings und Hash Maps, die es uns erlauben, variable Mengen von Daten zu speichern und zu verwalten. Lassen Sie uns diese Themen nun Schritt für Schritt und mit der gewohnten Gründlichkeit erkunden.

Teil 3: Enums, Pattern Matching und Collections

In diesem Teil erweitern wir unser Repertoire erheblich. Wir lernen, wie man Zustände und Varianten modelliert, wie man elegant auf unterschiedliche Datenstrukturen reagiert und wie man mit dynamischen Datenmengen umgeht.

Kapitel 11: Enums und Pattern Matching

Bisher haben wir Structs kennengelernt, um Daten zu gruppieren, die *zusammengehören*. Aber manchmal möchten wir einen Typ definieren, der einen Wert aus einer *Menge möglicher Varianten* annehmen kann. Hierfür sind **Enums** (Enumerations, Aufzählungen) das richtige Werkzeug. Gepaart mit Rusts **Pattern Matching** (`match` und `if let`) ermöglichen Enums sehr sauberen und sicheren Code zur Handhabung verschiedener Zustände oder Typen.

- **11.1 Enums definieren (Aufzählungen)**

Ein Enum wird mit dem Schlüsselwort `enum` definiert, gefolgt vom Namen (Konvention: `PascalCase`) und den möglichen Varianten innerhalb geschweifter Klammern `{ }`. Jede Variante ist selbst ein gültiger Wert des Enum-Typs.

- Rust

```
// Ein einfaches Enum, das vier Richtungen repräsentiert
enum Direction {
    Up,
    Down,
    Left,
    Right,
}
```

```
// Ein Enum, das verschiedene Ereignisse in einer GUI darstellen könnte
enum GuiEvent {
    Click { x: i32, y: i32 }, // Variante mit benannten Feldern (wie ein Struct)
    KeyPress(char),           // Variante mit einem Tupel-artigen Wert (char)
    WindowResize { width: u32, height: u32 },
    Quit,                     // Variante ohne zugehörige Daten
}

fn main() {
    let player_direction = Direction::Left; // Instanz des Direction-Enums
    let clicked_at = GuiEvent::Click { x: 100, y: 250 }; // Instanz von GuiEvent
    let key_pressed = GuiEvent::KeyPress('w');
    let quit_event = GuiEvent::Quit;

    process_direction(player_direction);
    process_event(clicked_at);
    process_event(key_pressed);
    process_event(quit_event);
}

fn process_direction(dir: Direction) {
    // Wir werden gleich sehen, wie man das mit 'match' eleganter macht
    // Hier nur zur Demonstration der Enum-Nutzung
    // ...
}

fn process_event(event: GuiEvent) {
    // ... Verarbeitung mit 'match' ...
}
```

-
- Wichtig ist: Eine Variable vom Typ `Direction` kann *entweder* `Direction::Up`, *oder* `Direction::Down`, *oder* `Direction::Left`, *oder* `Direction::Right` sein – niemals mehrere gleichzeitig. Ebenso für `GuiEvent`.

• 11.2 Daten mit Enum-Varianten verknüpfen

Wie im `GuiEvent`-Beispiel gesehen, können Enum-Varianten Daten speichern:

- **Keine Daten:** Wie `GuiEvent::Quit` oder alle Varianten von `Direction`.
- **Tupel-ähnliche Daten:** Wie `GuiEvent::KeyPress(char)`. Man gibt den Typ der Daten in Klammern an. Instanziiert wird mit `EnumName::VariantName(wert)`.
- **Struct-ähnliche Daten:** Wie `GuiEvent::Click { x: i32, y: i32 }`. Man gibt benannte Felder mit Typen in geschweiften Klammern an. Instanziiert wird ähnlich wie bei Structs: `EnumName::VariantName { feld1: wert1, ... }`.

- Dies macht Enums extrem flexibel, um verschiedene Arten von Informationen unter einem gemeinsamen Typ zu vereinen.

• 11.3 Der `Option<T>`-Enum

Eines der wichtigsten Enums in Rust ist `Option<T>`, das Teil der Standardbibliothek ist und so häufig verwendet wird, dass seine Varianten (`Some` und `None`) ohne `Option::`-Präfix genutzt werden können. Es ist Rusts Antwort auf das Problem von `null` oder `nil` in anderen Sprachen.

Die Definition sieht (vereinfacht) so aus:

- Rust

```
enum Option<T> {
    None, // Repräsentiert die Abwesenheit eines Wertes
    Some(T), // Repräsentiert die Anwesenheit eines Wertes vom Typ T
}
```

- - `T` ist ein generischer Typparameter (mehr dazu in Teil 5). `Option<i32>` ist ein Typ, der entweder `None` oder `Some(i32)` sein kann. `Option<String>` ist entweder `None` oder `Some(String)`.
 - **Der Clou:** Rust hat keinen speziellen `null`-Wert. Wenn ein Wert optional ist (also möglicherweise nicht vorhanden), *muss* der Typ dies explizit durch `Option<T>` ausdrücken. Der Compiler *zwingt* Sie dann, beide Fälle (`Some` und `None`) zu behandeln, bevor Sie versuchen können, den potenziell vorhandenen Wert zu verwenden. Dies eliminiert zur Kompilierzeit ganze Klassen von Fehlern, die in anderen Sprachen durch unerwartete `null`-Werte zur Laufzeit entstehen (z.B. `NullPointerException` in Java).

- Rust

```
fn main() {
    let some_number = Some(5); // Typ: Option<i32>
    let some_string = Some(String::from("ein String")); // Typ: Option<String>

    let absent_number: Option<i32> = None; // Typ: Option<i32>

    // Wie greifen wir auf den Wert '5' in 'some_number' zu?
    // Nicht direkt! Wir müssen prüfen, ob 'Some' oder 'None' vorliegt.
    // Das geht am besten mit 'match' (siehe nächster Punkt).

    // Beispiel: Funktion, die optional etwas zurückgibt
    let result1 = find_character('a', "rust ist toll");
```

```

let result2 = find_character('z', "rust ist toll");

println!("Position von 'a': {:?}", result1); // Gibt Some(7) aus
println!("Position von 'z': {:?}", result2); // Gibt None aus
}

// Gibt die Position des ersten Vorkommens eines Zeichens zurück, oder None
fn find_character(c: char, s: &str) -> Option<usize> {
    for (index, character) in s.char_indices() {
        if character == c {
            return Some(index); // Zeichen gefunden, Position zurückgeben
        }
    }
    None // Zeichen nicht gefunden
}

```

-
- **Analogie:** `Option<T>` ist wie eine Schachtel. Sie kann entweder leer sein (`None`) oder etwas vom Typ `T` enthalten (`Some(T)`). Bevor Sie etwas aus der Schachtel nehmen können, müssen Sie nachsehen, ob überhaupt etwas drin ist und was es ist.

• 11.4 Der `match`-Kontrollflussoperator

`match` ist Rusts mächtiges Werkzeug für Pattern Matching. Es erlaubt Ihnen, einen Wert mit einer Serie von Mustern (Patterns) zu vergleichen und Code basierend auf dem passenden Muster auszuführen. Es ist wie `if/else`, aber viel flexibler und sicherer, insbesondere im Zusammenspiel mit Enums.

Die Struktur ist:

- Rust

```

match wert {
    muster1 => { /* Code für Fall 1 */ },
    muster2 => { /* Code für Fall 2 */ },
    // ... weitere Muster
    _ => { /* Code für alle anderen Fälle (optional, aber oft nötig für Exhaustivität) */ },
}

```

-
- **Wichtige Eigenschaften von `match`:**
 - **Exhaustivität:** `match` ist *exhaustive*. Das bedeutet, Sie *müssen* jeden möglichen Fall (jede Variante eines Enums, jeden möglichen Wert) abdecken. Der Compiler prüft dies und gibt einen Fehler aus, wenn ein

Fall fehlt. Dies verhindert, dass Sie versehentlich einen Zustand vergessen.

- **Muster:** Die linken Seiten (`muster1`, `muster2`) sind Muster. Das können Literale, Variablennamen (die den Wert binden), Enum-Varianten, Structs, `_` (der Wildcard-Platzhalter) und komplexere Kombinationen sein.
- **Ausdruck:** Wie `if` ist auch `match` ein Ausdruck. Der Wert des ausgeführten Arms (`=> { ... }`) ist der Wert des gesamten `match`-Ausdrucks. Alle Arme müssen denselben Typ zurückgeben (oder alle divergieren, z.B. durch `panic!`).

- Rust

```
// Beispiel mit Option<T>
fn plus_one(x: Option<i32>) -> Option<i32> {
    match x {
        None => None, // Wenn x None ist, gib None zurück
        Some(i) => Some(i + 1), // Wenn x Some(i) ist, binde den Wert an i und gib Some(i+1)
    } // Exhaustiv: None und Some(T) sind alle Möglichkeiten für Option<T>
}
```

```
// Beispiel mit unserem GuiEvent-Enum
fn process_event_detailed(event: GuiEvent) {
    match event {
        GuiEvent::Click { x, y } => { // Destrukturiert die Felder x und y
            println!("Klick bei Koordinaten: x={}, y={}", x, y);
        }
        GuiEvent::KeyPress(c) => { // Bindet den char an c
            println!("Taste gedrückt: '{}'", c);
        }
        GuiEvent::WindowResize { width, height } => { // Destrukturiert width und height
            println!("Fenstergröße geändert: Breite={}, Höhe={}", width, height);
        }
        GuiEvent::Quit => { // Einfacher Varianten-Match
            println!("Beenden-Ereignis empfangen.");
        }
    }
    // Kein '_' nötig, da wir alle vier Varianten von GuiEvent behandelt haben.
}
```

```
fn main() {
    let five = Some(5);
    let six = plus_one(five); // six ist Some(6)
    let none = plus_one(None); // none ist None
    println!("six: {:?}, none: {:?}", six, none);

    process_event_detailed(GuiEvent::Click { x: 10, y: 20 });
}
```

```

process_event_detailed(GuiEvent::KeyPress('q'));
process_event_detailed(GuiEvent::Quit);
}

```

-
-

• 11.5 Muster, die Werte binden

Wie in den `match`-Beispielen gesehen, können Muster nicht nur zum Vergleichen dienen, sondern auch Teile des geprüften Werts an neue Variablen binden, die dann im Code-Block des jeweiligen `match`-Arms verfügbar sind.

- `Some(i)`: Prüft, ob der Wert ein `Some` ist, und bindet den enthaltenen Wert an die Variable `i`.
- `GuiEvent::Click { x, y }`: Prüft, ob es die `Click`-Variante ist, und bindet die Werte der Felder `x` und `y` an gleichnamige Variablen.
- `GuiEvent::KeyPress(c)`: Prüft, ob es die `KeyPress`-Variante ist, und bindet den enthaltenen `char` an die Variable `c`.

• 11.6 `if let` für einfache Fälle

Manchmal interessiert man sich in einem `match` nur für *einen einzigen* Fall und möchte alle anderen Fälle ignorieren oder gleich behandeln. Dafür kann `match` etwas umständlich sein. Rust bietet hierfür die `if let`-Syntax als praktische Abkürzung.

`if let muster = wert { ... }` ist äquivalent zu `match wert { muster => { ... }, _ => {} }`.

- Rust

```

fn main() {
    let Lieblingsfarbe: Option<&str> = None;
    let ist_sonntag = false;
    let alter: Option<u8> = Some(30);

    // Beispiel 1: Nur an 'alter' interessiert, wenn es Some ist
    if let Some(a) = alter { // Prüft, ob 'alter' ein Some ist, bindet Wert an 'a'
        println!("Das Alter aus Some ist: {}", a);
    } else { // Optionaler else-Block für alle anderen Fälle (hier nur None)
        println!("Kein Alter angegeben.");
    }

    // Beispiel 2: Nur wenn 'Lieblingsfarbe' Some("blau") ist
    if let Some("blau") = Lieblingsfarbe {
        println!("Die Lieblingsfarbe ist Blau!");
    } else if ist_sonntag {

```

```

    println!("Sonntags ist die Farbe Grün!");
} else {
    println!("Verwende eine andere Farbe.");
}

// Ohne 'if let' wäre Beispiel 1 umständlicher:
match alter {
    Some(a) => println!("Das Alter aus Some ist: {}", a),
    None => println!("Kein Alter angegeben."),
}
}

```

-
- `if let` ist besonders nützlich für `Option` und `Result` (das wir bei der Fehlerbehandlung kennenlernen), wenn man nur den "Erfolgsfall" (`Some` oder `Ok`) behandeln möchte. Es verliert jedoch die Exhaustivitätsprüfung des Compilers, die `match` bietet.

Kapitel 12: Kontrollfluss II - Schleifen

Schleifen werden verwendet, um einen Codeblock mehrmals auszuführen. Rust bietet drei Arten von Schleifenkonstrukten: `loop`, `while` und `for`.

• 12.1 Endlosschleife mit `loop`

Die `loop`-Schleife führt einen Codeblock unendlich oft aus, bis sie explizit mit dem Schlüsselwort `break` beendet wird.

- Rust

```

fn main() {
    let mut counter = 0;

    let result = loop { // 'loop' ist auch ein Ausdruck
        counter += 1;
        println!("Counter: {}", counter);

        if counter == 10 {
            break counter * 2; // Beendet die Schleife und gibt 'counter * 2' zurück
        }
        // Ohne 'break' würde diese Schleife ewig laufen.
    }; // Das Ergebnis von 'break' wird 'result' zugewiesen.

    println!("Schleife beendet. Ergebnis: {}", result); // Gibt 20 aus
}

```

-

- `loop` ist nützlich für Szenarien, bei denen man eine Operation wiederholen möchte, deren Abbruchbedingung komplexer ist und vielleicht erst innerhalb des Schleifenkörpers geprüft wird. Die Möglichkeit, mit `break` einen Wert zurückzugeben, ist oft praktisch.

- **12.2 Bedingte Schleifen mit `while`**

Eine `while`-Schleife führt ihren Codeblock aus, solange eine Bedingung (die ein `bool` ergeben muss) zu Beginn jeder Iteration `true` ist.

- Rust

```
fn main() {
    let mut number = 3;

    // Solange 'number' nicht 0 ist...
    while number != 0 {
        println!("{}", number);
        number -= 1; // Wichtig: Bedingung muss irgendwann false werden!
    }

    println!("LIFTOFF!!!");
}
```

-
- Die Bedingung wird *vor* der Ausführung des Blocks geprüft. Wenn die Bedingung von Anfang an `false` ist, wird der Block gar nicht erst ausgeführt.

- **12.3 Schleifen über Collections mit `for`**

Die `for`-Schleife ist die gebräuchlichste und oft sicherste Methode, um über die Elemente einer Sammlung (wie Arrays, Vektoren, Slices, Ranges oder andere Typen, die das `Iterator`-Trait implementieren) zu iterieren.

- Rust

```
fn main() {
    let a = [10, 20, 30, 40, 50];

    // Iteriert über *Referenzen* auf die Elemente des Arrays
    println!("Iterieren über Array (immutable):");
    for element in a.iter() { // a.iter() erzeugt einen Iterator über &i32
        println!("Der Wert ist: {}", element);
    }
}
```

```
// Iterieren über einen Range (Bereich)
// (1..4) ist exklusiv 4, also 1, 2, 3
println!("\nIterieren über Range (1..4):");
for number in 1..4 {
    println!("{}", number);
}

// (1..=4) ist inklusiv 4, also 1, 2, 3, 4
println!("\nIterieren über Range (1..=4):");
for number in 1..=4 {
    println!("{}", number);
}
}
```

- Die `for`-Schleife ist sicherer als manuelle `while`-Schleifen mit Index, da sie das Risiko von Indexfehlern (Zugriff außerhalb der Grenzen) vermeidet. Sie ist auch oft lesbarer. Der Ausdruck nach `in` (z.B. `a.iter()`, `1..4`) muss etwas sein, das in einen **Iterator** umgewandelt werden kann. Iteratoren sind ein Kernkonzept in Rust, das wir implizit nutzen; sie stellen eine Sequenz von Werten bereit.
 - `collection.iter()` gibt einen Iterator über unveränderliche Referenzen (`&T`).
 - `collection.iter_mut()` gibt einen Iterator über veränderliche Referenzen (`&mut T`), falls die Collection mutable ist.
 - `collection.into_iter()` übernimmt die Ownership der Collection und gibt einen Iterator über die Werte selbst (`T`).

Kapitel 13: Collections I - Vektoren

Arrays sind nützlich, aber ihre feste Größe ist oft einschränkend. Wenn wir eine Sammlung benötigen, deren Größe sich zur Laufzeit ändern kann, verwenden wir **Vektoren**.

• 13.1 Dynamische Listen mit `Vec<T>`

`Vec<T>` (aus `std::vec::Vec`, aber so häufig, dass `Vec` meist reicht) ist ein **dynamisch dimensionierbarer Vektor**, oft einfach als "Vektor" bezeichnet. Er kann wachsen und schrumpfen.

- `T` ist der Typ der Elemente, die der Vektor speichert. Alle Elemente müssen vom selben Typ `T` sein.
- Die Daten eines Vektors werden auf dem **Heap** gespeichert. Der Vektor selbst (der auf dem Stack liegt) verwaltet einen Zeiger auf die

Heap-Daten, seine aktuelle Länge und seine Kapazität (wie viel Platz auf dem Heap reserviert ist, bevor neu allokiert werden muss).

- Da `Vec<T>` Heap-Speicher verwaltet, unterliegt er den **Ownership-Regeln** (Move-Semantik, keine `Copy-Trait`).

● 13.2 Vektoren erstellen, aktualisieren und lesen

- Rust

```
fn main() {
    // Erstellen eines leeren Vektors (Typ muss oft annotiert werden)
    let mut v1: Vec<i32> = Vec::new();

    // Erstellen mit dem `vec!`-Makro (bequemer, Typ wird oft inferiert)
    let mut v2 = vec![1, 2, 3]; // Typ Vec<i32>

    // Aktualisieren (Hinzufügen von Elementen) - Vektor muss 'mut' sein!
    v1.push(5); // v1 ist jetzt [5]
    v1.push(6); // v1 ist jetzt [5, 6]
    v1.push(7); // v1 ist jetzt [5, 6, 7]

    v2.push(4); // v2 ist jetzt [1, 2, 3, 4]
    println!("v1: {:?}", v1);
    println!("v2: {:?}", v2);

    // Lesen von Elementen - Zwei Wege:

    // 1. Indexing `[]`: Gibt eine Referenz zurück (&T oder &mut T).
    //    Panic!, wenn der Index außerhalb der Grenzen liegt.
    let third: &i32 = &v2[2]; // Index 2 ist das dritte Element (Wert 3)
    println!("Das dritte Element von v2 ist {}", third);

    // let does_not_exist = &v2[100]; // <-- Würde panic!en!

    // 2. `get()`-Methode: Gibt eine `Option<&T>` zurück.
    //    Sicherer, da kein Panic, gibt `None` bei ungültigem Index zurück.
    let second: Option<&i32> = v2.get(1); // Index 1 ist das zweite Element (Wert 2)
    match second {
        Some(value) => println!("Das zweite Element von v2 (via get) ist {}", value),
        None => println!("Index 1 existiert nicht."),
    }

    let does_not_exist_get = v2.get(100);
    println!("Element bei Index 100: {:?}", does_not_exist_get); // Gibt None aus

    // Wichtige Borrowing-Regel bei Vektoren:
    let mut v = vec![100, 32, 57];
    let first = &v[0]; // Unveränderliche Referenz auf das erste Element
```

```
// v.push(64); // FEHLER! cannot borrow `v` as mutable because it is also borrowed as
immutable
    // Warum? .push() könnte eine Neuallokation auf dem Heap auslösen,
    // wodurch der Speicherort von v[0] ungültig würde und 'first'
    // zu einer dangling reference werden könnte. Rust verhindert das!

println!("Das erste Element ist {}", first); // Erst nachdem 'first' nicht mehr verwendet wird,
    // wäre ein v.push() wieder erlaubt.
} // v1 und v2 gehen hier aus dem Scope. Da sie Owner ihrer Heap-Daten sind,
    // wird ihr Speicher automatisch freigegeben (`drop` wird aufgerufen).
```

-
-

• 13.3 Iterieren über Vektor-Elemente

Die `for`-Schleife ist der bevorzugte Weg, um Vektoren zu durchlaufen.

- Rust

```
fn main() {
    let v = vec![100, 32, 57];
    println!("Iterieren über unveränderliche Referenzen:");
    for i in &v { // Äquivalent zu v.iter()
        println!("{}", i);
    }

    let mut v_mut = vec![10, 20, 30];
    println!("Iterieren über veränderliche Referenzen:");
    for i in &mut v_mut { // Äquivalent zu v_mut.iter_mut()
        *i += 10; // Dereferenzieren (*) um den Wert zu ändern
        println!("{}", i); // Gibt 20, 30, 40 aus
    }
    println!("v_mut nachher: {:?}", v_mut); // Ist jetzt [20, 30, 40]
}
```

-
-

• 13.4 Enums zum Speichern verschiedener Typen in Vektoren nutzen

Ein `Vec<T>` kann nur Elemente vom Typ `T` speichern. Wenn Sie eine Sammlung benötigen, die scheinbar unterschiedliche Typen enthält, können Sie ein Enum definieren, das diese Typen als Varianten umschließt. Der Vektor speichert dann Instanzen dieses Enums.

- Rust

```
enum SpreadsheetCell {
```

```

    Int(i32),
    Float(f64),
    Text(String),
}

fn main() {
    let row = vec![
        SpreadsheetCell::Int(3),
        SpreadsheetCell::Text(String::from("blau")),
        SpreadsheetCell::Float(10.12),
        SpreadsheetCell::Int(-5),
    ]; // Typ ist Vec<SpreadsheetCell>

    println!("Tabellenzeile:");
    for cell in &row {
        match cell {
            SpreadsheetCell::Int(i) => println!(" - Integer: {}", i),
            SpreadsheetCell::Float(f) => println!(" - Float: {}", f),
            SpreadsheetCell::Text(s) => println!(" - Text: {}", s),
        }
    }
}

```

- Dies ist ein häufiges Muster, um Heterogenität in Sammlungen zu erreichen, während die Typsicherheit von Rust gewahrt bleibt.

Kapitel 14: Collections II - Strings

Wir haben Strings (`String` und `&str`) bereits mehrfach angetroffen. Dieses Kapitel vertieft unser Verständnis, insbesondere im Hinblick auf ihren internen Aufbau (UTF-8) und die gebräuchlichsten Operationen.

• 14.1 Rusts Umgang mit Text (`String` vs. `&str`)

Lassen Sie uns die beiden Haupttypen für Text noch einmal klar gegenüberstellen:

- `String`:
 - Ein **owned** (besitzender) Typ.
 - Speichert Textdaten als eine Sequenz von Bytes auf dem **Heap**.
 - Die Bytes repräsentieren **UTF-8** kodierten Text.
 - Ist **dynamisch dimensionierbar** (kann wachsen und schrumpfen).

- Ist **mutierbar** (veränderbar), wenn die Variable mit `mut` deklariert ist.
- Wird oft durch `String::from("...")` oder `"...".to_string()` erstellt.
- Unterliegt der **Move-Semantik** (kein `Copy`).
- **Analogie:** Ein Word-Dokument auf Ihrer Festplatte, das Sie besitzen und bearbeiten können.
- `&str` (**String-Slice**):
 - Ein **borrowed** (geliehener) Typ – eine Referenz.
 - Zeigt auf eine Sequenz von UTF-8 Bytes, die *irgendwo anders* gespeichert sind (z.B. in einer `String`-Variablen auf dem Heap oder direkt im Programmcode für String-Literale).
 - Hat eine **feste Größe** (die Länge des Teils, auf den er zeigt).
 - Ist **immutable** (unveränderlich) – man kann den Text *durch* den Slice nicht ändern (obwohl der zugrundeliegende `String` evtl. änderbar ist, aber nicht während der Slice existiert, siehe Borrowing-Regeln).
 - String-Literale (wie `"Hallo"`) im Code sind vom Typ `&'static str` (ein Slice, der für die gesamte Programmlaufzeit gültig ist).
 - Implementiert `Copy`.
 - **Analogie:** Ein Zitat oder ein Lesezeichen in einem Buch. Sie können das Zitat lesen, aber nicht das Buch selbst ändern, nur weil Sie das Zitat haben.
- **Deref Coercion:** Eine wichtige Eigenschaft ist, dass Rust oft automatisch eine Referenz auf einen `String` (`&String`) in einen String-Slice (`&str`) umwandeln kann, wenn eine Funktion `&str` erwartet. Das nennt sich Deref Coercion und macht die Arbeit mit beiden Typen nahtloser. `fn print_slice(s: &str)` kann sowohl mit `print_slice("literal")` als auch mit `let my_string = String::from("owned"); print_slice(&my_string);` aufgerufen werden.

● 14.2 Strings erstellen und aktualisieren

● Rust

```
fn main() {
    // Erstellen
    let s1 = String::new(); // Leerer String
    let s2 = String::from("Initialer Inhalt");
    let s3 = "Anderer Inhalt".to_string();
    let s4 = format!("Zahl: {}, Text: {}", 42, "Hallo"); // Wie println!, aber gibt String zurück

    println!("s1: '{}', s2: '{}', s3: '{}', s4: '{}'", s1, s2, s3, s4);

    // Aktualisieren (String muss 'mut' sein)
```

```

let mut s = String::from("foo");
println!("Vorher: {}", s);

// Anhängen eines String-Slices (&str) - bevorzugt, leiht nur aus
s.push_str("bar");
println!("Nach push_str: {}", s); // "foobar"

// Anhängen eines einzelnen Zeichens (char)
s.push('!');
println!("Nach push: {}", s); // "foobar!"

// Konkatenation mit '+' Operator
// fn add(self, s: &str) -> String;
// Nimmt Ownership von 'self' (dem String links), hängt einen &str an, gibt *neuen* String
// zurück.
let s5 = String::from("Hallo, ");
let s6 = String::from("Welt!");
let s7 = s5 + &s6; // s5 wird hierher *verschoben* und ist danach ungültig! s6 wird nur geliehen
// (&s6).
// println!("{}", s5); // Fehler! s5 wurde verschoben.
println!("{}", s6); // s6 ist noch gültig.
println!("{}", s7); // "Hallo, Welt!"

// Die `format!`-Makro ist oft lesbarer und nimmt keine Ownership
let s8 = String::from("tic");
let s9 = String::from("tac");
let s10 = String::from("toe");
let game = format!("{}", s8, s9, s10); // s8, s9, s10 bleiben gültig
println!("{}", game); // "tic-tac-toe"
println!("s8 immer noch da: {}", s8);
}

```

-
-
- **14.3 UTF-8 und Indizierung von Strings (die Herausforderungen)**

Rusts Strings sind immer UTF-8 kodiert. Das bedeutet:

- Ein `char` kann 1 bis 4 Bytes belegen. `'A'` ist 1 Byte, `'ü'` ist 2 Bytes, `'😊'` ist 4 Bytes.
- Die Länge eines Strings (`.len()`) gibt die Anzahl der **Bytes**, nicht die Anzahl der Zeichen zurück.
- Direkte Indizierung mit `[]` (z.B. `&s[0]`) greift auf **Bytes** zu. Dies ist erlaubt, aber gefährlich, wenn man nicht sicher ist, dass der Index auf den Anfang eines gültigen UTF-8-Zeichens zeigt. Ein Index mitten in einem Mehrbyte-Zeichen führt zu einem **panic**.

- Rust

```
fn main() {
    let hallo = String::from("Здравствуй"); // Russisch "Hallo"
    println!("{}", Länge (Bytes): {}, hallo, hallo.len()); // Gibt 24 Bytes aus

    // println!("{}", &hallo[0]); // Fehler: Kann kein Byte als Zeichen drucken
    // println!("{}", &hallo[1]); // Fehler und potenziell Panic!, da Index 1 mitten im ersten Zeichen
    // liegt.

    // Sicherer Weg: Iterieren über Zeichen oder Bytes
}
```

-
- Aus diesem Grund erlaubt Rust *keine* direkte Indizierung, um ein `char` zu erhalten (z.B. `s[i]`), weil dies in UTF-8 keine O(1)-Operation (konstante Zeit) wäre, wie man es von Indizierung oft erwartet (man müsste potenziell den String von vorne durchgehen, um das i-te Zeichen zu finden). Stattdessen muss man explizit über Zeichen oder Bytes iterieren.

• 14.4 Methoden zur String-Manipulation

Die `String`- und `&str`-Typen bieten viele nützliche Methoden. Einige Beispiele:

- `replace(from, to)`: Ersetzt alle Vorkommen von `from` durch `to` (gibt neuen String zurück).
- `trim()`: Entfernt führende und nachfolgende Leerzeichen (gibt `&str` zurück).
- `split_whitespace()`: Gibt einen Iterator über Wörter zurück, getrennt durch Leerzeichen.
- `contains(pattern)`: Prüft, ob der String ein Muster enthält.
- `starts_with(pattern)`, `ends_with(pattern)`: Prüfen Anfang/Ende.
- Konsultieren Sie die [Standardbibliothek-Dokumentation](#) für eine vollständige Liste.
- 14.5 Iterieren über Strings

Die sicheren Wege, die Inhalte eines Strings zu durchlaufen:

- Rust

```
fn main() {
    let s = "नमस्ते"; // Hindi "Hallo"

    println!("Iterieren über Chars:");
    for c in s.chars() { // Gibt einen Iterator über 'char' zurück
        println!("{}", c); // Gibt 'न', 'म', 'स्', 'ते' einzeln aus
    }
}
```

```
println!("\nIterieren über Bytes:");
for b in s.bytes() { // Gibt einen Iterator über 'u8' (Bytes) zurück
    println!("{}", b); // Gibt die numerischen Byte-Werte aus (18 Bytes insgesamt)
}
}
```

-
- `.chars()` ist meistens das, was man will, wenn man logische Zeichen verarbeiten muss. `.bytes()` ist nützlich für Low-Level-Verarbeitung oder wenn man mit ASCII-kompatiblen Protokollen arbeitet.

Kapitel 15: Collections III - Hash Maps

Die letzte Collection, die wir in diesem Teil betrachten, ist die **Hash Map**. Sie speichert Daten als Sammlung von **Schlüssel-Wert-Paaren** (Key-Value Pairs).

- **15.1 Schlüssel-Wert-Paare mit** `HashMap<K, V>`

`HashMap<K, V>` (aus `std::collections::HashMap`) speichert eine Zuordnung von eindeutigen Schlüsseln (`K`) zu Werten (`V`).

- Sie ist wie ein Wörterbuch oder Dictionary in anderen Sprachen.
- Die Daten werden auf dem **Heap** gespeichert.
- Sie bietet schnellen Zugriff (im Durchschnitt $O(1)$) auf einen Wert, wenn man den Schlüssel kennt. Dies wird durch eine **Hashing-Funktion** erreicht, die den Schlüssel in einen Index umwandelt, um den Speicherort des Werts zu bestimmen.
- Daher müssen alle Schlüssel (`K`) die Traits `Eq` (für Gleichheitsvergleiche) und `Hash` (für die Hashing-Berechnung) implementieren. Viele Standardtypen wie `String`, `i32`, `u64`, etc. tun dies bereits.
- Die Reihenfolge der Elemente in einer `HashMap` ist **nicht garantiert**.
- **15.2 Hash Maps erstellen, darauf zugreifen und aktualisieren**

- Rust

```
// Importieren der HashMap
use std::collections::HashMap;

fn main() {
    // Erstellen einer leeren HashMap
    let mut scores: HashMap<String, i32> = HashMap::new();

    // Einfügen von Schlüssel-Wert-Paaren mit 'insert'
```

```

// Beachten Sie: Schlüssel und Werte werden per Move hineinbewegt (wenn nicht Copy)
let team_blue = String::from("Blau");
let team_yellow = String::from("Gelb");

scores.insert(team_blue, 10); // Ownership von team_blue wird an scores übergeben
scores.insert(team_yellow, 50);

// println!("{}", team_blue); // Fehler! team_blue wurde verschoben.

println!("Scores: {:?}", scores);

// Zugriff auf Werte

// 1. `get(&key)`: Gibt `Option<&V>` zurück (sicher)
let blue_score: Option<i32> = scores.get(&String::from("Blau")); // Schlüssel muss geliehen
werden
match blue_score {
    Some(score) => println!("Score Blau (get): {}", score), // Gibt 10 aus
    None => println!("Team Blau nicht gefunden."),
}

// 2. Indexing `[]`: Gibt `&V` zurück, aber panic!, wenn Schlüssel nicht existiert.
// Nur für lesenden Zugriff empfohlen, wenn man sicher ist, dass der Schlüssel existiert.
// let score_blaue_index = scores[&String::from("Blau")]; // Vorsicht, funktioniert hier, aber
riskant
// let score_rot_index = scores[&String::from("Rot")]; // <-- Würde panic!en!

// Iterieren über Schlüssel-Wert-Paare (Reihenfolge nicht garantiert)
println!("Iterieren über Scores:");
for (key, value) in &scores { // Leih sich die Map unveränderlich
    println!("{}", key, value);
}

// Aktualisieren von Werten

// Einfaches Überschreiben mit 'insert'
scores.insert(String::from("Blau"), 25); // Überschreibt den alten Wert 10
println!("Scores nach Update Blau: {:?}", scores);

// Einfügen nur, wenn Schlüssel noch nicht existiert: `entry().or_insert()`
// `entry()` gibt ein `Entry`-Enum zurück (Occupied oder Vacant)
// `or_insert()` fügt den Wert ein, wenn Vacant, und gibt eine *mutable Referenz* zurück.
scores.entry(String::from("Gelb")).or_insert(100); // Gelb existiert, Wert bleibt 50
scores.entry(String::from("Rot")).or_insert(75); // Rot existiert nicht, wird eingefügt (75)

println!("Scores nach or_insert: {:?}", scores);

// Wert basierend auf altem Wert aktualisieren
let text = "Hallo Welt wunderbare Welt";
let mut word_counts = HashMap::new();

```

```

for word in text.split_whitespace() {
    // `or_insert(0)` gibt &mut i32 zurück (entweder auf neuen Wert 0 oder alten Wert)
    let count = word_counts.entry(word.to_string()).or_insert(0);
    *count += 1; // Dereferenzieren und Inkrementieren
}
println!("\nWortzählungen: {:?}", word_counts);
// Gibt z.B. {"Welt": 2, "wunderbare": 1, "Hallo": 1} aus (Reihenfolge kann variieren)
}

```

-
-

• 15.3 Ownership bei Hash Maps

Wie im Beispiel `scores.insert(team_blue, 10);` gesehen:

- Für Typen, die nicht `Copy` sind (wie `String`), wird die **Ownership** des Schlüssels und des Wertes an die `HashMap` **übertragen**, wenn sie mit `insert` eingefügt werden. Die ursprünglichen Variablen sind danach ungültig.
- Für Typen, die `Copy` sind (wie `i32`), werden die Werte **kopiert**, und die Originale bleiben gültig.
- Wenn Sie Referenzen als Schlüssel oder Werte speichern wollen, müssen Sie sich um deren **Lifetimes** (Gültigkeitsdauern) kümmern, um sicherzustellen, dass die Referenzen nicht länger leben als die Daten, auf die sie zeigen. Dies ist ein fortgeschrittenes Thema.

Damit haben wir Teil 3 abgeschlossen! Sie haben nun ein solides Verständnis von **Enums** zur Modellierung von Varianten, dem mächtigen **Pattern Matching** mit `match` und `if let`, den verschiedenen **Schleifenkonstrukten** und den drei grundlegenden **Collections**: dem dynamischen Array `Vec<T>`, den Texttypen `String`/`&str` mit ihren UTF-8-Besonderheiten und der Schlüssel-Wert-Sammlung `HashMap<K, V>`.

Diese Werkzeuge sind essentiell für das Schreiben von realistischem Rust-Code. Sie ermöglichen es Ihnen, komplexere Datenstrukturen und Logiken auszudrücken und Sammlungen von Daten effizient zu verwalten.

Teil 4: Fehlerbehandlung und Code-Organisation

Sehr gut, wir nähern uns den fortgeschritteneren, aber äußerst wichtigen Aspekten des robusten und organisierten Programmierens in Rust. Willkommen zu **Teil 4: Fehlerbehandlung und Code-Organisation**. In diesem Abschnitt werden wir uns eingehend mit Rusts differenziertem Ansatz zur Fehlerbehandlung beschäftigen, der zwischen behebbaren und unbehebbaren Fehlern unterscheidet. Anschließend lernen wir, wie wir unseren Code mithilfe des Modulsystems strukturieren und wie wir externe Bibliotheken über das Cargo-Ökosystem einbinden und verwalten. Diese Fähigkeiten sind unerlässlich, um größere, wartbare und widerstandsfähige Anwendungen zu entwickeln.

Teil 4: Fehlerbehandlung und Code-Organisation

Nachdem wir nun wissen, wie wir Datenstrukturen definieren und grundlegende Logik implementieren, konzentrieren wir uns darauf, wie wir mit Fehlern umgehen und wie wir unseren Code übersichtlich und wiederverwendbar gestalten.

Kapitel 16: Fehlerbehandlung I - Unbehebbarer Fehler mit `panic!`

Nicht alle Fehler sind gleich. Rust unterscheidet grundlegend zwischen Fehlern, von denen sich ein Programm potenziell erholen kann, und Fehlern, die einen so schwerwiegenden Zustand anzeigen, dass eine Fortsetzung nicht sinnvoll oder sicher ist. Für letztere Kategorie ist das `panic!`-Makro vorgesehen.

- **16.1 Was ist `panic!`?**

Das `panic!`-Makro signalisiert, dass im Programm ein **unbehebbarer Fehler** aufgetreten ist. Wenn `panic!` aufgerufen wird, geschieht Folgendes:

- Das Programm gibt eine Fehlermeldung aus, die oft den Ort des Panics (Dateiname, Zeilennummer) und eine optionale Nachricht enthält.
- Es beginnt mit dem **Stack Unwinding**: Der aktuelle Thread arbeitet sich den Aufrufstapel (Stack) rückwärts hoch und räumt die Daten für jede angetroffene Funktion auf (insbesondere ruft er die `drop`-Methode für alle lokalen Variablen auf, um Ressourcen wie Speicher oder Dateihandles freizugeben).
- Schließlich wird der Thread beendet. Wenn es der Hauptthread ist, wird das gesamte Programm beendet.

- Alternativ kann Rust so konfiguriert werden, dass es bei einem Panic sofort abbricht (`abort`), ohne den Stack abzuwickeln. Dies führt zu einer kleineren Binärdatei, gibt aber keine Ressourcen sauber frei. Unwinding ist das Standardverhalten.

Analogie: Stellen Sie sich `panic!` wie das Ziehen der Notbremse in einem Zug vor. Es ist eine drastische Maßnahme für eine Notsituation, die den normalen Betrieb sofort und abrupt beendet, idealerweise aber auf eine kontrollierte Weise (Unwinding), um weiteren Schaden zu vermeiden.

- **16.2 Wann `panic!` verwenden?**

`panic!` sollte **ausschließlich für unbehebbarer Fehler** verwendet werden, die typischerweise auf einen **Programmierfehler (Bug)** hindeuten oder einen Zustand repräsentieren, von dem sich das Programm vernünftigerweise nicht erholen kann oder sollte. Situationen, in denen `panic!` angebracht sein *kann*:

- **Verletzung von Zusicherungen (Assertions):** Wenn eine Bedingung, die *immer* wahr sein sollte (eine Invariante), verletzt wird. Beispiel: Zugriff auf einen Array-Index außerhalb der Grenzen (`a[10]` bei einem Array der Größe 5). Dies signalisiert fast immer einen Logikfehler im Code.
- **Unerreichbare Zustände:** Wenn der Code einen Zustand erreicht, der logisch unmöglich sein sollte.
- **Fehler in Beispielen oder Tests:** Um klar zu signalisieren, dass etwas Unerwartetes passiert ist, das untersucht werden muss.
- **Unmittelbar nach einer Operation, von der man weiß, dass sie nicht fehlschlagen kann:** Wenn man z.B. gerade erfolgreich einen Wert in eine `HashMap` eingefügt hat und ihn direkt danach wieder abrufen, *könnte* man `unwrap()` (was bei `None` paniced) verwenden, da ein Fehlschlag hier einen Bug in der `HashMap`-Implementierung oder der eigenen Logik bedeuten würde. Aber selbst hier ist Vorsicht geboten.
- **Vermeiden Sie `panic!` für erwartete Fehler!** Wenn eine Operation fehlschlagen *kann* (z.B. das Öffnen einer Datei, die nicht existiert; das Parsen einer Benutzereingabe, die ungültig ist; eine Netzwerkanfrage, die fehlschlägt), dann sollte `Result<T, E>` verwendet werden (siehe nächstes Kapitel). Die aufrufende Funktion soll die Möglichkeit haben, auf diese erwarteten Fehler zu reagieren. Ein `panic!` nimmt ihr diese Möglichkeit. Ein gutes Bibliotheksdesign vermeidet `panic!` in seiner öffentlichen API für vorhersehbare Fehlerfälle.

- **16.3 Backtraces bei Panics**

Wenn ein `panic!` auftritt, kann Rust einen **Backtrace** (oder Stack Trace)

ausgeben. Dies ist eine Liste der Funktionen, die aufgerufen wurden, in der Reihenfolge ihres Aufrufs, bis hin zur Stelle, an der der Panic ausgelöst wurde. Backtraces sind extrem wertvoll für die Fehlersuche, da sie zeigen, wie das Programm an den Punkt des Fehlers gelangt ist.

Um einen vollständigen Backtrace zu erhalten, müssen Sie möglicherweise eine Umgebungsvariable setzen, bevor Sie Ihr Programm ausführen:

- Bash

```
RUST_BACKTRACE=1 cargo run
```

-
- (In Debug-Builds ist ein einfacher Backtrace oft standardmäßig aktiviert, RUST_BACKTRACE=1 oder RUST_BACKTRACE=full liefert mehr Details).

Die Ausgabe sieht dann typischerweise so aus (Beispiel):

```
thread 'main' panicked at 'index out of bounds: the len is 3 but the index is 5', src/main.rs:4:5
stack backtrace:
```

```
0: rust_begin_unwind
   at
/rustc/e1884a8e3c3e813aada8254edfa1dac5680795d4/library/std/src/panicking.rs:517:5
1: core::panicking::panic_fmt
   at
/rustc/e1884a8e3c3e813aada8254edfa1dac5680795d4/library/core/src/panicking.rs:101:14
2: core::panicking::panic_bounds_check
   at
/rustc/e1884a8e3c3e813aada8254edfa1dac5680795d4/library/core/src/panicking.rs:77:5
3: <usize as core::slice::index::SliceIndex<[i32]>>::index
   at
/rustc/e1884a8e3c3e813aada8254edfa1dac5680795d4/library/core/src/slice/index.rs:184:10
4: core::slice::index::<impl core::ops::index::Index<usize> for [T]>::index
   at
/rustc/e1884a8e3c3e813aada8254edfa1dac5680795d4/library/core/src/slice/index.rs:170:9
5: <[i32] as core::ops::index::Index<usize>>::index
   at
/rustc/e1884a8e3c3e813aada8254edfa1dac5680795d4/library/core/src/slice/mod.rs:2782:9
6: my_program::main
   at ./src/main.rs:4:5
7: core::ops::function::FnOnce::call_once
   at
/rustc/e1884a8e3c3e813aada8254edfa1dac5680795d4/library/core/src/ops/function.rs:227:5
note: Some details are omitted, run with `RUST_BACKTRACE=full` for a verbose backtrace.
```

-

- Zeile 6 (`my_program::main at ./src/main.rs:4:5`) zeigt uns genau, wo in unserem Code der Fehler (hier ein Indexzugriff in Zeile 4, Spalte 5 von `main.rs`) aufgetreten ist.

Kapitel 17: Fehlerbehandlung II - Behebbarer Fehler mit `Result<T, E>`

Für die meisten Fehler, die in einem Programm auftreten können – insbesondere solche, die durch externe Faktoren wie ungültige Eingaben, nicht verfügbare Ressourcen oder Netzwerkprobleme verursacht werden – ist ein `panic!` unangemessen. Stattdessen signalisieren wir diese **behebbarer Fehler** mit dem `Result<T, E>`-Enum, sodass der aufrufende Code entscheiden kann, wie er darauf reagieren möchte.

• 17.1 Der `Result<T, E>`-Enum

Ähnlich wie `Option<T>` ist `Result<T, E>` ein Enum aus der Standardbibliothek, das zwei Zustände repräsentiert: Erfolg oder Fehlschlag.

Die Definition sieht (vereinfacht) so aus:

- Rust

```
enum Result<T, E> {
    Ok(T), // Repräsentiert einen Erfolg und enthält den Ergebniswert vom Typ T
    Err(E), // Repräsentiert einen Fehler und enthält einen Fehlerwert vom Typ E
}
```

- - `T` ist der Typ des Wertes bei Erfolg.
 - `E` ist der Typ des Wertes bei einem Fehler. `E` sollte Informationen über den aufgetretenen Fehler enthalten.
 - Wie `Option` erzwingt `Result` die explizite Behandlung beider Fälle (`Ok` und `Err`) zur Kompilierzeit. Man kann einen Fehler nicht einfach ignorieren.

- Rust

```
use std::fs::File;
use std::io::Read;

fn main() {
    // File::open gibt Result<File, std::io::Error> zurück
    let file_result = File::open("nicht_existente_datei.txt");
```

```
// Wir MÜSSEN prüfen, ob es Ok oder Err war
match file_result {
    Ok(file_handle) => {
        println!("Datei erfolgreich geöffnet: {:?}", file_handle);
        // ... hier könnten wir mit der Datei arbeiten ...
    }
    Err(error) => {
        println!("Fehler beim Öffnen der Datei: {:?}", error);
        // Hier könnten wir z.B. eine Standardkonfiguration laden,
        // den Benutzer informieren oder das Programm beenden.
    }
}
}
```

- **Analogie:** Eine Funktion, die `Result<T, E>` zurückgibt, ist wie ein Paketbote. Er bringt Ihnen entweder das erwartete Paket (`Ok(T)`) oder einen Abholschein mit einer Begründung, warum es nicht geklappt hat (`Err(E)`), z.B. "Adresse nicht gefunden" oder "Empfänger nicht angetroffen". Sie müssen den Zettel lesen, um zu wissen, was passiert ist.
- **17.2 Umgang mit Result-Werten** (`match`, `unwrap`, `expect`)

Es gibt verschiedene Möglichkeiten, mit einem `Result`-Wert umzugehen:

- `match` (**Der sicherste Weg**): Erlaubt die explizite Behandlung beider Fälle.
- Rust

```
let result: Result<i32, String> = Ok(10); // Nur als Beispiel
match result {
    Ok(value) => println!("Erfolg! Wert ist: {}", value),
    Err(e) => println!("Fehler aufgetreten: {}", e),
}
```

-
-
- `unwrap()` (**Der riskante Weg**): Gibt den Wert aus `Ok(T)` zurück. Wenn das `Result` jedoch `Err(E)` ist, ruft `unwrap()` `panic!` auf.

- **Verwendung:** Nur wenn Sie aufgrund der Programmlogik *absolut sicher* sind, dass das `Result` immer `Ok` sein *muss*. Ein `Err` würde in diesem Fall einen Bug darstellen. In Bibliotheken oder unsicheren Kontexten stark vermeiden!

- Rust

```
// let value = File::open("wichtige_config.cfg").unwrap(); // Paniced, wenn Datei nicht da!
let value = Ok(5).unwrap(); // Gibt 5 zurück
println!("Unwrapped value: {}", value);
```

-
-
- `expect(message)` (**Der etwas bessere riskante Weg**): Funktioniert wie `unwrap()`, aber im Falle eines `Err(E)` verwendet es die übergebene `message` als `panic!`-Nachricht. Das hilft bei der Fehlersuche.

- **Verwendung:** Gleiche Bedingungen wie `unwrap()`, aber bevorzugt, da die Fehlermeldung im Panic-Fall aussagekräftiger ist.

- Rust

```
// let value = File::open("wichtige_config.cfg")
//   .expect("Konnte wichtige_config.cfg nicht öffnen!"); // Bessere Panic-Meldung
let value = Ok(10).expect("Sollte Ok sein"); // Gibt 10 zurück
println!("Expected value: {}", value);
```

-
-
- **Andere Methoden:** `Result` hat viele nützliche Methoden wie `is_ok()`, `is_err()`, `ok()` (wandelt in `Option<T>` um), `err()` (wandelt in `Option<E>` um), `map()`, `map_err()`, `and_then()`, etc., die eine funktionalere Fehlerbehandlung ermöglichen.

● 17.3 Fehler weitergeben mit dem `?`-Operator

Sehr oft möchte man einen Fehler, der in einer aufgerufenen Funktion auftritt, nicht direkt an Ort und Stelle behandeln, sondern an die *eigene* aufrufende Funktion weitergeben. Dies manuell mit `match` zu tun, kann sehr umständlich werden:

- Rust

```
use std::fs::File;
use std::io::{self, Read};

// Manuelle Fehlerweitergabe mit match (umständlich)
fn read_username_from_file_manual() -> Result<String, io::Error> {
    let file_result = File::open("username.txt");

    let mut file = match file_result {
        Ok(f) => f,
```

```

    Err(e) => return Err(e), // Fehler sofort zurückgeben
};

let mut username = String::new();

match file.read_to_string(&mut username) {
    Ok(_) => Ok(username), // Erfolg zurückgeben
    Err(e) => Err(e),      // Fehler zurückgeben
}
}

```

-
- Der **?-Operator** vereinfacht dieses Muster drastisch. Wenn **?** auf einen **Result**-Wert angewendet wird:
 - Ist der Wert **Ok(T)**, wird **T** zurückgegeben und die Ausführung geht normal weiter.
 - Ist der Wert **Err(E)**, wird die aktuelle Funktion *sofort* beendet und das **Err(E)** wird zurückgegeben.
- **Wichtig:** Der **?-Operator** kann nur in Funktionen verwendet werden, deren Rückgabotyp selbst ein **Result<T, E>** (oder **Option<T>**) ist, wobei der Fehlertyp **E** kompatibel sein muss (oft durch automatische Konvertierung via **From**-Trait).
- Rust

```

// Gleiche Funktion mit dem ?-Operator (viel kürzer und idiomatischer)
fn read_username_from_file_idiomatic() -> Result<String, io::Error> {
    let mut file = File::open("username.txt"?); // Wenn Err, return Err(e)
    let mut username = String::new();
    file.read_to_string(&mut username)?; // Wenn Err, return Err(e)
    Ok(username) // Wenn alles gut ging, gib Ok(username) zurück
}

```

```

// Noch kürzer durch Verkettung:
fn read_username_from_file_chained() -> Result<String, io::Error> {
    let mut username = String::new();
    File::open("username.txt")?.read_to_string(&mut username)?;
    Ok(username)
}

```

```

// Oder noch direkter mit fs::read_to_string:
use std::fs;
fn read_username_from_file_stdlib() -> Result<String, io::Error> {
    fs::read_to_string("username.txt")
}

```

```

fn main() {
    match read_username_from_file_idiomatic() {

```

```

Ok(name) => println!("Benutzername (idiomatic): {}", name),
Err(e) => println!("Fehler (idiomatic): {}", e),
}
// ... Aufrufe der anderen Varianten ...
}

```

- Der `?`-Operator macht die Fehlerbehandlung in Rust sehr ergonomisch, indem er das häufige Muster der Fehlerweitergabe vereinfacht, ohne die Explizitheit von `Result` aufzugeben.

• 17.4 Wann `Result` statt `panic!` verwenden?

Faustregel:

- Verwenden Sie `Result<T, E>` für alle Fehler, die **erwartbar** sind und von denen sich das Programm oder der aufrufende Code potenziell **erholen** kann (z.B. Datei nicht gefunden, Netzwerkfehler, ungültige Eingabe, Parsing-Fehler). Dies ist der Standard für die meisten Fehler in robusten Anwendungen und Bibliotheken.
- Verwenden Sie `panic!` (direkt oder indirekt über `unwrap/expect`) für Fehler, die auf einen **Programmierfehler (Bug)** hindeuten oder einen Zustand signalisieren, der **niemals auftreten sollte** und von dem eine sinnvolle Fortsetzung **unmöglich** oder unsicher ist (z.B. Index außerhalb der Grenzen, Verletzung von Invarianten).

Kapitel 18: Module und das Modulsystem

Wenn Projekte wachsen, wird es unerlässlich, den Code zu organisieren, um ihn übersichtlich, wartbar und wiederverwendbar zu halten. Rusts Modulsystem dient genau diesem Zweck. Es ermöglicht:

- **Namespacing:** Vermeidung von Namenskollisionen.
- **Sichtbarkeitskontrolle:** Definition einer öffentlichen API und Verbergen interner Implementierungsdetails (Encapsulation).
- **Strukturierung:** Aufteilung des Codes in logische Einheiten.
- **18.1 Code in Module aufteilen (`mod`)**

Das Schlüsselwort `mod` deklariert ein neues Modul. Module können verschachtelt werden.

- Rust

// In src/main.rs oder src/lib.rs

```
mod front_of_house { // Definiert ein Modul 'front_of_house'
    // Inhalt ist standardmäßig privat für front_of_house

    pub mod hosting { // Definiert ein *öffentliches* Untermodul 'hosting'
        // Inhalt ist standardmäßig privat für hosting

        pub fn add_to_waitlist() {} // *Öffentliche* Funktion in hosting

        fn seat_at_table() {} // *Private* Funktion in hosting
    }

    mod serving { // Definiert ein *privates* Untermodul 'serving'
        fn take_order() {}
        fn serve_order() {}
        fn take_payment() {}
    }
} // Ende mod front_of_house

// Funktion außerhalb des Moduls
pub fn eat_at_restaurant() {
    // Zugriff auf öffentliche Elemente über vollständigen Pfad
    // Pfade starten mit 'crate' (für den Crate-Root) oder Modulnamen

    // Absoluter Pfad von crate root
    crate::front_of_house::hosting::add_to_waitlist();

    // Relativer Pfad (wenn 'eat_at_restaurant' im selben Modul wie 'front_of_house' wäre)
    // front_of_house::hosting::add_to_waitlist();

    // Fehler! seat_at_table ist privat
    // crate::front_of_house::hosting::seat_at_table();

    // Fehler! serving ist privat
    // crate::front_of_house::serving::take_order();
}
```

-
-
- **18.2 Pfade zu Modulen** (`use`, `super`, `self`)

Um auf Elemente in Modulen zuzugreifen, verwendet man Pfade, die durch `::` getrennt sind.

1. **Absolute Pfade:** Beginnen mit `crate` (dem Wurzelmodul des aktuellen Crates) oder dem Namen eines externen Crates. Beispiel:
`crate::front_of_house::hosting::add_to_waitlist, std::collections::HashMap.`
 2. **Relative Pfade:** Beginnen mit `self` (dem aktuellen Modul), `super` (dem Elternmodul) oder einem Bezeichner im aktuellen Modul. Beispiel:
`self::some_function, super::sibling_module::some_item.`
- Das ständige Schreiben langer Pfade ist mühsam. Das `use`-Schlüsselwort bringt Pfade in den aktuellen Gültigkeitsbereich (Scope), sodass man kürzere Namen verwenden kann.

- Rust

```
// Bringt das Modul 'hosting' in den Scope
use crate::front_of_house::hosting;
// Bringt eine spezifische Funktion in den Scope
use crate::front_of_house::hosting::add_to_waitlist;
// Bringt Typen wie Structs, Enums oder Traits oft direkt in den Scope
use std::collections::HashMap;
// Importiert alles aus einem Modul (Glob-Import) - mit Vorsicht verwenden!
use std::io::*;

pub fn eat_at_restaurant_with_use() {
    // Nach 'use crate::front_of_house::hosting;'
    hosting::add_to_waitlist(); // Kürzerer Pfad

    // Nach 'use crate::front_of_house::hosting::add_to_waitlist;'
    add_to_waitlist(); // Direkter Aufruf

    let mut map = HashMap::new(); // Direkte Verwendung nach 'use'
    map.insert(1, 2);
}

mod customer {
    use crate::front_of_house::hosting; // Geht auch in Untermodulen
    // Verwendung von 'super' um auf 'eat_at_restaurant_with_use' im Elternmodul zuzugreifen
    use super::eat_at_restaurant_with_use;

    fn order_food() {
        hosting::add_to_waitlist();
        eat_at_restaurant_with_use(); // Zugriff via 'use super::... '
    }
}
```

-
- **Konventionen für `use`:**

1. Für Funktionen: Es ist oft üblich, das *Modul* in den Scope zu bringen und die Funktion mit `modul::funktion()` aufzurufen (z.B. `hosting::add_to_waitlist()`). Das macht klarer, woher die Funktion stammt.
2. Für Structs, Enums, Traits: Es ist üblich, den Typ *direkt* in den Scope zu bringen (z.B. `use std::collections::HashMap;`).
3. Ausnahme: Wenn zwei Typen denselben Namen haben, bringt man das Modul oder verwendet `use ... as ...` zur Umbenennung: `use std::fmt::Result; use std::io::Result as IoResult;`

● 18.3 Sichtbarkeit kontrollieren (`pub`)

Standardmäßig sind alle Elemente (Module, Funktionen, Structs, Enums, Konstanten) in einem Modul **privat**. Sie können nur innerhalb des Moduls selbst und seiner Kindmodule verwendet werden.

Das Schlüsselwort `pub` macht ein Element **öffentlich**, sodass es von Code außerhalb des Moduls verwendet werden kann.

● Rust

```
mod my_module {
    pub fn public_function() { // Diese Funktion ist von außen sichtbar
        println!("Aufruf von public_function()");
        private_function(); // Kann private Funktion im selben Modul aufrufen
    }

    fn private_function() { // Diese Funktion ist nur innerhalb von my_module sichtbar
        println!("Aufruf von private_function()");
    }

    pub struct PublicStruct { // Struct ist öffentlich
        pub public_field: i32, // Feld ist öffentlich
        private_field: i32, // Feld ist privat (Standard)
    }

    impl PublicStruct {
        pub fn new(value: i32) -> PublicStruct { // Öffentlicher Konstruktor
            PublicStruct {
                public_field: value,
                private_field: 10,
            }
        }
        // Methoden sind auch privat, wenn nicht 'pub'
        pub fn print_private(&self){
            println!("Privates Feld: {}", self.private_field);
        }
    }
}
```

```

pub enum PublicEnum { // Enum ist öffentlich
    VariantA, // Varianten eines öffentlichen Enums sind automatisch öffentlich
    VariantB(String),
}
} // Ende my_module

fn main() {
    my_module::public_function();
    // my_module::private_function(); // Fehler! private_function ist privat

    let s = my_module::PublicStruct::new(5);
    println!("Öffentliches Feld: {}", s.public_field); // OK
    // println!("Privates Feld: {}", s.private_field); // Fehler! private_field ist privat
    s.print_private(); // OK, Methode ist public

    let e = my_module::PublicEnum::VariantB(String::from("Hallo")); // OK
}

```

- Die Sichtbarkeitsregeln helfen dabei, die interne Implementierung eines Moduls zu kapseln und eine stabile öffentliche Schnittstelle (API) zu definieren.

• 18.4 Dateien für Module verwenden

Um zu vermeiden, dass alle Module in einer einzigen riesigen Datei landen, kann man Module in separate Dateien auslagern. Rust hat klare Regeln dafür, wo es nach dem Code für ein Modul sucht:

1. Szenario 1: `mod my_module;` in `src/main.rs` oder `src/lib.rs`
Der Compiler sucht nach dem Inhalt von `my_module` in:

- `src/my_module.rs` ODER
- `src/my_module/mod.rs`

2. Szenario 2: `mod my_module;` in `src/some_other_module.rs`
Der Compiler sucht nach dem Inhalt von `my_module` in:

- `src/some_other_module/my_module.rs` ODER
- `src/some_other_module/my_module/mod.rs`

• Beispiel Dateistruktur:

```

my_project/
├── Cargo.toml
└── src/

```

```

└─ main.rs      // Enthält: mod utils; fn main() { utils::do_stuff(); }
└─ utils.rs     // Enthält den Code für das 'utils'-Modul ODER:
└─ utils/       // Alternativer Ordner für 'utils'
    └─ mod.rs   // Enthält Code für 'utils', evtl. mit 'pub mod helpers;'
    └─ helpers.rs // Enthält Code für das Untermodul 'helpers'

```

-
- In `main.rs` steht nur `mod utils;` (plus `fn main()`). Der eigentliche Code für `utils` steht entweder in `utils.rs` oder in `utils/mod.rs`. Wenn `utils/mod.rs` weitere Untermodule wie `pub mod helpers;` deklariert, sucht der Compiler nach deren Code in `utils/helpers.rs`.

Kapitel 19: Crates und Cargo

Wir haben Module innerhalb eines Projekts betrachtet. Nun heben wir den Blick auf die nächsthöhere Ebene: **Crates** und **Pakete**, verwaltet durch **Cargo**.

• 19.1 Was ist ein Crate? (Binary vs. Library)

Ein **Crate** ist die kleinste Kompilierungseinheit in Rust. Wenn der Compiler `rustc` aufgerufen wird, verarbeitet er genau einen Crate. Ein Crate kann in zwei Hauptformen auftreten:

- **Binary Crate (Binär-Crate):** Kompiliert zu einer **ausführbaren Datei**. Muss eine `fn main()` als Einstiegspunkt haben. Der Quellcode dafür liegt typischerweise in `src/main.rs`. Wird mit `cargo new projektname` erstellt.
- **Library Crate (Bibliotheks-Crate):** Kompiliert zu wiederverwendbarem Code (einer Bibliothek), der von anderen Crates (Binär- oder Bibliotheks-Crates) als Abhängigkeit genutzt werden kann. Hat keine `fn main()`. Der Quellcode liegt typischerweise in `src/lib.rs`. Wird mit `cargo new --lib bibliotheksname` erstellt.

• 19.2 Pakete und Crates

Ein **Paket (Package)** ist eine Sammlung von einem oder mehreren Crates, die zusammen eine bestimmte Funktionalität bereitstellen. Jedes Paket wird durch eine `Cargo.toml`-Datei beschrieben. Ein Paket:

- Muss **mindestens einen Crate** enthalten (entweder Bibliothek oder Binär).
- Kann **höchstens einen Library Crate** enthalten (`src/lib.rs`).
- Kann **beliebig viele Binary Crates** enthalten. Zusätzliche Binär-Crates legt man in das Verzeichnis `src/bin/`. Jede Datei in `src/bin/` (z.B. `src/bin/mein_tool.rs`) wird als separater Binary Crate mit eigenem `fn main()` behandelt und kann mit `cargo run --bin mein_tool` ausgeführt werden.

- Cargo ist das Werkzeug, das Pakete verwaltet. Wenn Sie `cargo build` ausführen, kompiliert Cargo den oder die Crates, die zu Ihrem Paket gehören.
- **19.3 Externe Crates von `crates.io` verwenden (`Cargo.toml`)**

Einer der größten Vorteile von Rust ist sein Ökosystem und der Paketmanager Cargo. Auf `crates.io`, dem offiziellen Crate-Repository der Rust-Community, finden Sie Tausende von Open-Source-Bibliotheken (Crates), die Sie in Ihren eigenen Projekten verwenden können.

Um einen externen Crate zu verwenden:

- Finden Sie den Crate auf `crates.io` (z.B. den Crate `rand` für Zufallszahlen).
- Öffnen Sie die `Cargo.toml`-Datei Ihres Pakets.
- Fügen Sie unter dem Abschnitt `[dependencies]` eine Zeile mit dem Namen des Crates und der gewünschten Version hinzu (gemäß den Angaben auf `crates.io`):
- Ini, TOML

```
[package]
name = "mein_projekt"
version = "0.1.0"
edition = "2021"
```

See more keys and their definitions at <https://doc.rust-lang.org/cargo/reference/manifest.html>

```
[dependencies]
rand = "0.8.5" # Fügt den 'rand' Crate als Abhängigkeit hinzu
# Hier könnten weitere Abhängigkeiten stehen
# z.B. regex = "1.5"
```

-
-
- Wenn Sie nun Cargo-Befehle wie `cargo build` oder `cargo check` ausführen, wird Cargo:
 - Die angegebene Version des externen Crates (und dessen Abhängigkeiten) von `crates.io` herunterladen.
 - Den externen Crate kompilieren.
 - Ihren Code kompilieren und gegen den externen Crate linken.
- Sie können Funktionen, Structs etc. aus dem externen Crate in Ihrem Code mit `use` verwenden, wobei der Crate-Name der Start des Pfades ist:
- Rust

```
use rand::Rng; // Importiert das Rng-Trait aus dem 'rand'-Crate
```

```
fn main() {  
    let mut rng = rand::thread_rng();  
    let n: u32 = rng.gen_range(1..=100); // Verwendet Funktionalität aus 'rand'  
    println!("Zufallszahl: {}", n);  
}
```

- Cargo kümmert sich um das Herunterladen, Kompilieren und Verlinken von Abhängigkeiten und macht die Wiederverwendung von Code extrem einfach und robust.

- **19.4** `cargo build`, `cargo run`, `cargo check`

Lassen Sie uns die Hauptbefehle von Cargo im Kontext von Paketen und Abhängigkeiten nochmals zusammenfassen:

- `cargo check`: Überprüft schnell, ob Ihr Paket (und seine Abhängigkeiten) kompiliert, ohne eine Binärdatei zu erzeugen. Ideal für schnelles Feedback während der Entwicklung. Lädt und kompiliert Abhängigkeiten beim ersten Mal oder bei Änderungen.
- `cargo build`: Kompiliert Ihr Paket (und seine Abhängigkeiten). Erzeugt eine nicht optimierte Binärdatei (oder Bibliothek) in `target/debug/`.
- `cargo build --release`: Kompiliert Ihr Paket (und seine Abhängigkeiten) mit Optimierungen. Erzeugt eine optimierte Binärdatei (oder Bibliothek) in `target/release/`. Dies dauert länger, führt aber zu deutlich schnellerem Code.
- `cargo run`: Kompiliert (falls nötig, wie `cargo build`) und führt dann den Haupt-Binary-Crate (`src/main.rs`) Ihres Pakets aus. Wenn Sie mehrere Binaries haben (in `src/bin/`), können Sie mit `cargo run --bin <name>` ein spezifisches auswählen.
- Cargo ist das zentrale Werkzeug für das gesamte Rust-Projektmanagement, von der Erstellung über die Kompilierung und Abhängigkeitsverwaltung bis hin zum Testen (`cargo test`), Dokumentieren (`cargo doc`) und Veröffentlichen (`cargo publish`).

Damit haben wir Teil 4 abgeschlossen. Sie haben nun ein tiefgreifendes Verständnis von Rusts zweigleisiger Fehlerbehandlungsstrategie mit `panic!` für Bugs und `Result<T, E>` für behebbare Fehler, einschließlich des mächtigen `?`-Operators. Außerdem kennen Sie die Grundlagen des Modulsystems zur Strukturierung Ihres Codes und

die Rolle von Cargo und Crates für das Management von Paketen und Abhängigkeiten.

Diese Konzepte sind entscheidend für das Schreiben von Code, der nicht nur funktioniert, sondern auch robust, wartbar und gut organisiert ist.

Teil 5: Weiterführende Konzepte (Einführung)

Guten Tag! Es ist jetzt Montag, der 14. April 2025, 16:21 Uhr hier in Koblenz. Wir haben bereits eine beachtliche Strecke auf unserer Lernreise durch Rust zurückgelegt und solide Fundamente in den Grundlagen, dem Ownership-System, den Collections und der Fehlerbehandlung geschaffen. Nun ist es an der Zeit, uns **Teil 5: Weiterführende Konzepte (Einführung)** zuzuwenden.

Dieser letzte Teil unseres Grundkurses führt Sie in einige der mächtigsten Abstraktionsmechanismen von Rust ein: **Generics**, um wiederverwendbaren Code zu schreiben, **Traits**, um gemeinsames Verhalten zu definieren, und **Lifetimes**, um die Gültigkeit von Referenzen sicherzustellen. Außerdem werfen wir einen Blick auf das essenzielle Thema des automatisierten Testens und schließen mit einem Vorschlag für ein kleines Projekt, in dem Sie das Gelernte anwenden können. Auch wenn diese Themen komplexer erscheinen mögen, sind sie doch integraler Bestandteil dessen, was Rust so leistungsfähig und sicher macht. Wir werden sie mit der gewohnten Sorgfalt und Gründlichkeit untersuchen.

Teil 5: Weiterführende Konzepte (Einführung)

In diesem Abschnitt lernen wir Techniken kennen, um unseren Code flexibler, wiederverwendbarer und abstrakter zu gestalten, die Sicherheit von Referenzen zu gewährleisten und die Korrektheit unserer Programme durch Tests zu überprüfen.

Kapitel 20: Generics - Wiederverwendbaren Code schreiben

Stellen Sie sich vor, Sie müssten eine Funktion schreiben, die das größte Element in einer Liste findet. Sie könnten eine Funktion für `i32`-Listen schreiben, eine für `f64`-Listen, eine für `String`-Listen usw. Das führt zu viel Code-Duplizierung. **Generics** (generische Typen) sind Rusts Lösung für dieses Problem. Sie ermöglichen es uns, Code zu schreiben, der mit abstrakten Typen arbeitet, die erst zur Kompilierzeit durch konkrete Typen ersetzt werden.

- **20.1 Generische Typen in Funktionsdefinitionen**

Wir können Funktionen definieren, die generische Typparameter verwenden. Diese Parameter deklarieren wir innerhalb von spitzen Klammern `<...>` direkt nach dem Funktionsnamen. Der Parameter dient als Platzhalter für einen beliebigen Typ.

- Rust

```

// Eine einfache Funktion, die einen Wert beliebigen Typs 'T'
// entgegennimmt und denselben Wert zurückgibt.
// '<T>' deklariert den generischen Typparameter T.
fn identity<T>(item: T) -> T {
    item
}

// Eine Funktion, die versucht, den größten Wert in einem Slice zu finden.
// ACHTUNG: Dieser Code kompiliert so noch NICHT! Wir sehen gleich warum.
/*
fn largest_incomplete<T>(list: &[T]) -> &T {
    let mut largest = &list[0];
    for item in list {
        // Fehler: Wie soll Rust wissen, wie man Typ 'T' vergleicht (>)?
        // if item > largest {
        //     largest = item;
        // }
    }
    largest
}
*/

fn main() {
    let num = identity(5); // T wird zu i32
    let character = identity('a'); // T wird zu char
    let boolean = identity(true); // T wird zu bool

    println!("Identität: num={}, char={}, bool={}", num, character, boolean);
}

```

-
- Im identity-Beispiel wird der Platzhalter T zur Kompilierzeit durch den konkreten Typ des übergebenen Arguments ersetzt (i32, char, bool). Das largest_incomplete-Beispiel zeigt ein Problem: Wenn wir mit generischen Typen arbeiten, weiß der Compiler nicht automatisch, welche Operationen auf diesen Typen erlaubt sind. Woher soll er wissen, dass man Werte vom Typ T mit > vergleichen kann? Hierfür benötigen wir Trait Bounds (siehe Kapitel 21).

• 20.2 Generische Typen in Struct- und Enum-Definitionen

Auch Structs und Enums können generisch definiert werden, um Datenstrukturen zu schaffen, die mit unterschiedlichen Typen arbeiten können. Wir haben dies bereits bei `Option<T>` und `Result<T, E>` gesehen.

- Rust

```

// Ein Punkt im 2D-Raum, dessen Koordinaten von einem beliebigen Typ T sein können.

```



```
struct Point<T> {
    x: T,
    y: T,
}
```

// Methoden können auch auf generischen Structs implementiert werden.

// '<T>' nach 'impl' deklariert, dass die Implementierung für jedes T gilt.

```
impl<T> Point<T> {
    // Diese Methode gibt eine Referenz auf das x-Feld zurück.
    // Sie funktioniert für jedes T.
    fn x(&self) -> &T {
        &self.x
    }
}
```

// Wir können auch Methoden *nur* für spezifische konkrete Typen implementieren.

// Hier: Eine Methode, die nur für Point<f32> existiert.

```
impl Point<f32> {
    fn distance_from_origin(&self) -> f32 {
        (self.x.powi(2) + self.y.powi(2)).sqrt()
    }
}
```

// Ein Enum, das entweder einen generischen Wert oder einen Fehler enthält.

// '<T, E>' deklariert zwei generische Typparameter.

```
enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

```
fn main() {
```

```
    let integer_point = Point { x: 5, y: 10 }; // T wird zu i32
```

```
    let float_point = Point { x: 1.0, y: 4.0 }; // T wird zu f64
```

```
    // let mixed_point = Point { x: 5, y: 4.0 }; // Fehler! x und y müssen den gleichen Typ T haben.
```

```
    println!("Integer Punkt x: {}", integer_point.x());
```

```
    println!("Float Punkt x: {}", float_point.x());
```

```
    // println!("Distanz Integer: {}", integer_point.distance_from_origin()); // Fehler! Methode
    // existiert nur für Point<f32>
```

```
    println!("Distanz Float: {}", float_point.distance_from_origin());
```

```
    let success: Result<String, &str> = Result::Ok(String::from("Erfolg!"));
```

```
    let failure: Result<String, &str> = Result::Err("Fehlgeschlagen");
```

```
}
```

-
- Generische Structs und Enums ermöglichen es uns, vielseitige und typsichere Datenstrukturen zu definieren.

- **20.3 Monomorphisierung**

Wie funktionieren Generics zur Laufzeit? Haben sie einen Performance-Nachteil? Nein! Rust implementiert Generics durch **Monomorphisierung** zur Kompilierzeit. Das bedeutet, der Compiler schaut sich an, mit welchen konkreten Typen Ihr generischer Code aufgerufen wird, und erzeugt für jede Kombination eine spezialisierte, nicht-generische Version des Codes.

- Wenn Sie `identity(5)` (mit `i32`) und `identity('a')` (mit `char`) aufrufen, erzeugt der Compiler im Hintergrund quasi zwei Funktionen: eine für `i32` und eine für `char`.
- Das Ergebnis ist, dass generischer Code genauso schnell läuft wie spezialisierter Code – es gibt keinen Laufzeit-Overhead für die generische Abstraktion. Der mögliche Nachteil ist, dass der kompilierte Code größer werden kann, da für jede verwendete Typkombination Code generiert wird.

Kapitel 21: Traits - Gemeinsames Verhalten definieren

Generics erlauben uns, über Typen zu abstrahieren. Aber was ist, wenn wir über **Verhalten** abstrahieren wollen? Was, wenn wir eine Funktion schreiben wollen, die mit jedem Typ funktioniert, der eine bestimmte Fähigkeit besitzt (z.B. sich selbst drucken, verglichen werden, serialisiert werden)? Hier kommen **Traits** ins Spiel.

- **21.1 Was sind Traits? (Ähnlich zu Interfaces)**

Ein Trait definiert eine Sammlung von Methoden (und manchmal assoziierten Typen oder Konstanten), die ein Typ implementieren muss, um als Träger dieses Traits zu gelten. Traits beschreiben also gemeinsames Verhalten oder eine Fähigkeit. Sie sind Rusts primärer Mechanismus für Ad-hoc-Polymorphie und ähneln Konzepten wie Interfaces in Java/C# oder Protokollen in Swift.

- **21.2 Traits definieren und implementieren**

Wir definieren einen Trait mit dem `trait`-Schlüsselwort, gefolgt vom Namen (Konvention: `PascalCase`) und den Methodensignaturen innerhalb von `{ }`.

- Rust

```
// Definition des Traits 'Summary'
trait Summary {
```

```

// Methodensignatur: Nimmt eine unveränderliche Referenz auf self
// und gibt einen String zurück.
fn summarize(&self) -> String;

// Traits können auch Default-Implementierungen bereitstellen
fn default_summary(&self) -> String {
    String::from("(Zusammenfassung nicht verfügbar)")
}
}

// Ein Struct für einen Tweet
pub struct Tweet {
    pub username: String,
    pub content: String,
    pub reply: bool,
    pub retweet: bool,
}

// Implementierung des 'Summary'-Traits FÜR den Typ 'Tweet'
impl Summary for Tweet {
    fn summarize(&self) -> String {
        format!("{: {}", self.username, self.content)
    }
    // Wir könnten hier auch default_summary überschreiben, tun es aber nicht.
}

// Ein Struct für einen Nachrichtenartikel
pub struct NewsArticle {
    pub headline: String,
    pub location: String,
    pub author: String,
    pub content: String,
}

// Implementierung des 'Summary'-Traits FÜR den Typ 'NewsArticle'
impl Summary for NewsArticle {
    fn summarize(&self) -> String {
        format!("{}, von {} {}", self.headline, self.author, self.location)
    }

    // Überschreiben der Default-Implementierung
    fn default_summary(&self) -> String {
        format!("Lesen Sie mehr: {}...", self.headline)
    }
}

fn main() {
    let tweet = Tweet {
        username: String::from("horse_ebooks"),
        content: String::from("natürlich, wie jeder weiß, Pferde"),
    }
}

```

```

    reply: false,
    retweet: false,
};

```

```

let article = NewsArticle {
    headline: String::from("Pinguine gewinnen die Eishockey-WM!"),
    location: String::from("Pittsburgh, PA, USA"),
    author: String::from("Iceburgh"),
    content: String::from("Die Pittsburgh Penguins sind wieder die besten..."),
};

```

```

println!("1 neuer Tweet: {}", tweet.summarize());
println!("Neuer Artikel: {}", article.summarize());
println!("Default Tweet: {}", tweet.default_summary()); // Nutzt Default aus Trait
println!("Default Artikel: {}", article.default_summary()); // Nutzt überschriebene Version
}

```

-
-

• 21.3 Traits als Parameter (Trait Bounds)

Die wahre Stärke von Traits zeigt sich, wenn wir sie als **Trait Bounds** (Trait-Beschränkungen) für generische Typparameter verwenden. Dies erlaubt uns, Funktionen zu schreiben, die jeden Typ akzeptieren, der einen bestimmten Trait implementiert.

Es gibt zwei äquivalente Syntaxformen:

- `impl Trait`-Syntax (einfacher):

- Rust

// Akzeptiert jeden Typ 'item', der das Trait 'Summary' implementiert.

```

pub fn notify_impl(item: &impl Summary) {
    println!("Breaking News! {}", item.summarize());
}

```

-
-

- Trait Bound-Syntax (traditionell, flexibler):

- Rust

// Dasselbe wie oben, aber expliziter mit generischem Parameter T
 // 'T: Summary' bedeutet "T muss das Trait Summary implementieren".

```
pub fn notify_bound<T: Summary>(item: &T) {
    println!("Breaking News! {}", item.summarize());
}
```

- Beide Funktionen (`notify_impl` und `notify_bound`) können sowohl mit einem `&Tweet` als auch mit einem `&NewsArticle` aufgerufen werden, da beide Typen `Summary` implementieren.
- Rust

```
# // Definitionen von Summary, Tweet, NewsArticle von oben...
# trait Summary { fn summarize(&self) -> String; }
# struct Tweet { username: String, content: String }
# impl Summary for Tweet { fn summarize(&self) -> String { format!("{}", self.username, self.content) } }
# struct NewsArticle { headline: String, author: String, location: String, content: String }
# impl Summary for NewsArticle { fn summarize(&self) -> String { format!("{}", self.headline, self.author, self.location) } }
# pub fn notify_impl(item: &impl Summary) { println!("Breaking News! {}", item.summarize()); }
# pub fn notify_bound<T: Summary>(item: &T) { println!("Breaking News! {}", item.summarize()); }
# fn main() {
# let tweet = Tweet { username: String::from("a"), content: String::from("b") };
# let article = NewsArticle { headline: String::from("c"), location: String::from("d"), author: String::from("e"), content: String::from("f") };
notify_impl(&tweet); // Funktioniert
notify_impl(&article); // Funktioniert
notify_bound(&tweet); // Funktioniert
notify_bound(&article); // Funktioniert
# }
```

- - **Mehrere Trait Bounds:** Mit `+` kann man fordern, dass ein Typ mehrere Traits implementiert: `fn process<T: Summary + std::fmt::Display>(item: &T)`
 - **where-Klausel:** Für komplexere Bounds kann eine `where`-Klausel die Lesbarkeit verbessern:
 - Rust

```
use std::fmt::{Debug, Display};
fn some_function<T, U>(t: &T, u: &U) -> i32
    where T: Display + Clone,
          U: Clone + Debug
{
    // ...
    0 // Dummy-Rückgabe
}
```

-
-
- **Zurück zu Generics:** Jetzt können wir unsere `largest`-Funktion korrekt implementieren! Wir benötigen einen Trait, der Vergleichsoperationen ermöglicht. `std::cmp::PartialOrd` ist ein solcher Trait für partielle Ordnung (erlaubt `<`, `>`, etc.).

- Rust

```
use std::cmp::PartialOrd; // Importieren des Traits
```

```
// T muss PartialOrd implementieren (und oft auch Copy oder Clone, je nach Implementierung)
```

```
fn largest<T: PartialOrd>(list: &[T]) -> &T {
```

```
    // Annahme: list ist nicht leer
```

```
    let mut largest = &list[0];
```

```
    for item in list {
```

```
        if item > largest { // > ist jetzt erlaubt, da T: PartialOrd
```

```
            largest = item;
```

```
        }
```

```
    }
```

```
    largest
```

```
}
```

```
# fn main() {
```

```
    let numbers = vec![34, 50, 25, 100, 65];
```

```
    println!("Die größte Zahl ist {}", largest(&numbers)); // T wird zu i32
```

```
    let chars = vec!['y', 'm', 'a', 'q'];
```

```
    println!("Der größte Char ist {}", largest(&chars)); // T wird zu char
```

```
# }
```

-
-
- 21.4 Traits - Rückgabetypen
Man kann auch Funktionen schreiben, die einen Typ zurückgeben, der einen bestimmten Trait implementiert, ohne den konkreten Typ preiszugeben:

- Rust

```
# trait Summary { fn summarize(&self) -> String; }
```

```
# struct Tweet { username: String, content: String }
```

```
# impl Summary for Tweet { fn summarize(&self) -> String { format!("{}", self.username, self.content) } }
```

```
fn returns_summarizable() -> impl Summary {
```

```
    // Hier geben wir einen Tweet zurück, aber der Aufrufer weiß nur,
```

```
// dass er *irgendetwas* bekommt, das 'Summary' implementiert.
Tweet {
    username: String::from("example"),
    content: String::from("some content"),
    // reply: false, // Felder aus vorheriger Definition entfernt für Kürze
    // retweet: false,
}
}
# fn main() {
let summary = returns_summarizable();
println!("Zusammenfassung: {}", summary.summarize());
# }
```

-
- Dies ist nützlich für Abstraktion und das Verbergen von Implementierungsdetails.

Kapitel 22: Lifetimes - Gültigkeitsdauern verstehen (Grundlagen)

Lifetimes (Lebensdauern) sind ein weiteres Kernkonzept von Rusts Borrow Checker, das sicherstellt, dass Referenzen immer gültig sind und niemals "dangling" (auf freigegebenen Speicher zeigend) werden. In vielen Fällen kann der Compiler Lifetimes selbstständig ableiten (**Elision**), aber manchmal, insbesondere in Funktionssignaturen mit Referenzen, müssen wir dem Compiler mit expliziten **Lifetime-Annotationen** helfen, die Beziehungen zwischen den Gültigkeitsdauern von Referenzen zu verstehen.

• 22.1 Warum Lifetimes gebraucht werden (bei Referenzen)

Das Hauptziel des Borrow Checkers ist es, sicherzustellen, dass Referenzen nicht länger leben als die Daten, auf die sie verweisen.

Betrachten wir eine Funktion, die die längere von zwei String-Referenzen zurückgibt:

- Rust

```
/* // Dieser Code kompiliert NICHT ohne Lifetime-Annotationen
fn longest(x: &str, y: &str) -> &str {
    if x.len() > y.len() {
        x
    } else {
        y
    }
}
*/
```

-
- Der Compiler weiß hier nicht, wie die Lebensdauer der zurückgegebenen Referenz mit den Lebensdauern der Eingabereferenzen `x` und `y` zusammenhängt. Ist die zurückgegebene Referenz so lange gültig wie `x`? Oder so lange wie `y`? Was ist, wenn `x` und `y` unterschiedliche Gültigkeitsdauern haben? Der Compiler kann die Sicherheit nicht garantieren und verlangt daher explizite Informationen. Die zurückgegebene Referenz *muss* an die *kürzere* der beiden Eingabe-Lifetimes gebunden sein, um sicherzustellen, dass sie nicht auf Daten zeigt, die möglicherweise bereits ungültig geworden sind.

• 22.2 Lifetime-Annotationen in Funktionssignaturen

Lifetime-Annotationen beschreiben die *Beziehungen* zwischen den Lifetimes von Referenzen, ändern aber nicht die tatsächliche Lebensdauer der Werte. Die Syntax verwendet ein Apostroph `'` gefolgt von einem Namen (typischerweise Kleinbuchstaben, beginnend mit `'a`, `'b`, etc.).

- Rust

```
// Korrekte Version der 'longest'-Funktion mit Lifetime-Annotationen
// <'a> deklariert einen generischen Lifetime-Parameter 'a.
// x: &'a str - x ist eine Referenz, die mindestens so lange lebt wie 'a.
// y: &'a str - y ist eine Referenz, die mindestens so lange lebt wie 'a.
// -> &'a str - Die zurückgegebene Referenz lebt ebenfalls mindestens so lange wie 'a.
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() {
        x
    } else {
        y
    }
}

fn main() {
    let string1 = String::from("langer String ist lang");
    let result;
    { // Innerer Scope beginnt
        let string2 = String::from("xyz");
        // 'longest' wird aufgerufen. Die konkrete Lifetime 'a wird die *kürzere*
        // der Lifetimes von string1 und string2 sein (also die des inneren Scopes).
        result = longest(string1.as_str(), string2.as_str());
        println!("Der längste String ist: {}", result); // OK, result ist hier gültig
    } // Innerer Scope endet, string2 wird gedroppt
    // println!("Der längste String ist: {}", result); // FEHLER! result (gebunden an 'a,
    //           // also an den inneren Scope) ist hier
    //           // nicht mehr gültig, da string2 nicht mehr existiert.
```


// Der Compiler verhindert diesen Fehler durch die Lifetimes!

}

-
- Die Annotation `longest<'a>(x: &'a str, y: &'a str) -> &'a str` sagt dem Compiler: "Die zurückgegebene Referenz stammt entweder von `x` oder `y`, daher muss sie so lange gültig sein wie die kürzeste Gültigkeitsdauer von `x` und `y` (repräsentiert durch `'a`)."

Lifetimes können auch in Struct-Definitionen benötigt werden, wenn der Struct Referenzen enthält:

- Rust

// Ein Struct, das eine Referenz auf einen String-Slice hält.

// Die Instanz von ImportantExcerpt kann nicht länger leben als die Referenz in 'part'.

```
struct ImportantExcerpt<'a> {  
    part: &'a str,  
}
```

```
# fn main() {  
    let novel = String::from("Call me Ishmael. Some years ago...");  
    let first_sentence = novel.split('.').next().expect("Could not find a '.");  
    let i = ImportantExcerpt {  
        part: first_sentence, // 'part' leiht sich 'first_sentence'  
    };  
    println!("Wichtiger Auszug: {}", i.part);  
# }
```

-

-

• 22.3 Grundlegende Regeln und Inferenz (Lifetime Elision Rules)

Glücklicherweise müssen wir nicht *jede* Referenz mit einer Lifetime annotieren. Der Rust-Compiler verwendet eine Reihe von **Lifetime Elision Rules** (Regeln zur Auslassung von Lebensdauern), um Lifetimes in einfachen, häufigen Mustern automatisch abzuleiten. Wenn der Code diesen Regeln folgt, können die Annotationen weggelassen werden.

Die drei Hauptregeln (vereinfacht):

1. Jeder Referenzparameter in einer Funktion erhält seine *eigene*, unterschiedliche Lifetime-Annotation. (`fn foo<'a, 'b>(x: &'a i32, y: &'b i32)`)
2. Wenn es genau *einen* Eingabe-Lifetime-Parameter gibt, wird diese Lifetime allen Ausgabe-Referenzen zugewiesen. (`fn foo<'a>(x: &'a i32) -> &'a i32`)

3. Wenn es mehrere Eingabe-Lifetime-Parameter gibt, aber einer davon `&self` oder `&mut self` ist (d.h., es ist eine Methode), wird die Lifetime von `self` allen Ausgabe-Referenzen zugewiesen.
- Nur wenn diese Regeln nicht eindeutig bestimmen können, wie die Lifetimes zusammenhängen (wie in unserem `longest`-Beispiel), müssen wir explizite Annotationen hinzufügen.

- **22.4 Die statische Lebensdauer ('static)**

Eine besondere Lifetime ist `'static`. Eine Referenz mit der `'static`-Lifetime ist für die *gesamte Dauer des Programms* gültig. Alle String-Literale haben implizit den Typ `&'static str`, da sie direkt in die Binärdatei des Programms einkompiliert werden und somit immer verfügbar sind.

- Rust

```
let s: &'static str = "Ich lebe für immer (im Programm).";
```

-
-

Lifetimes sind ein mächtiges Konzept, das die Speichersicherheit von Rust untermauert. Sie können anfangs verwirrend sein, aber die Elision-Regeln decken viele Fälle ab, und die expliziten Annotationen sind nur dort nötig, wo der Compiler Hilfe braucht, um die Beziehungen zwischen Referenzen zu verstehen.

Kapitel 23: Automatisierte Tests

Das Schreiben von Tests ist ein integraler Bestandteil der Softwareentwicklung, um sicherzustellen, dass der Code wie erwartet funktioniert und auch nach Änderungen korrekt bleibt (Regressionstests). Rust hat eine eingebaute Unterstützung für das Schreiben und Ausführen von automatisierten Tests, die eng mit Cargo integriert ist.

- **23.1 Testfunktionen schreiben (`#[test]`)**

Eine Testfunktion in Rust ist eine normale Funktion, die mit dem Attribut `#[test]` annotiert wird. Sie hat typischerweise keine Parameter und gibt nichts zurück. Tests werden oft in einem Untermodul namens `tests` platziert, das mit `#[cfg(test)]` markiert wird, sodass es nur kompiliert wird, wenn Tests ausgeführt werden (`cargo test`), aber nicht im normalen Build (`cargo build`).

- Rust

```
// Beispiel-Funktion, die wir testen wollen
pub fn add(left: usize, right: usize) -> usize {
    left + right
}

// Testmodul (wird nur bei 'cargo test' kompiliert)
#[cfg(test)]
mod tests {
    // Importiert die Funktion aus dem Elternmodul
    use super::*;

    // Eine Testfunktion, markiert mit #[test]
    #[test]
    fn it_works() {
        let result = add(2, 2);
        assert_eq!(result, 4); // Behauptung: result muss gleich 4 sein
    }

    #[test]
    fn another_test() {
        // Dieser Test würde fehlschlagen, wenn er nicht auskommentiert wäre
        // panic!("Dieser Test soll fehlschlagen");
        assert!(true); // Ein einfacher Test, der immer besteht
    }
}
```

-
-
- **23.2** `assert!`, `assert_eq!`, `assert_ne!`

Innerhalb von Testfunktionen verwenden wir **Assertions-Makros**, um zu überprüfen, ob Bedingungen erfüllt sind. Wenn eine Assertion fehlschlägt, `panic!` der Test, was als Fehlschlag gewertet wird.

- `assert!(bool_expression)`: Überprüft, ob der Ausdruck `true` ergibt. `assert!(5 > 3); // OK`
- `assert_eq!(left, right)`: Überprüft, ob `left == right` ist (mittels `PartialEq`-Trait). Bei Fehlschlag wird eine hilfreiche Meldung ausgegeben, die beide Werte zeigt. Dies ist oft `assert!` vorzuziehen, wenn man Gleichheit prüft. `assert_eq!(add(5, 3), 8); // OK`
- `assert_ne!(left, right)`: Überprüft, ob `left != right` ist. `assert_ne!(add(2, 2), 5); // OK`
- **23.3 Tests mit `cargo test` ausführen**

Der Befehl `cargo test` im Terminal führt alle Tests im Projekt aus:

- Er kompiliert den Code im Testmodus (inklusive `#[cfg(test)]`-Module).
- Er erstellt eine Test-Binärdatei.

- Er führt die Test-Binärdatei aus, die alle Funktionen mit `#[test]` parallel ausführt (standardmäßig).
- Er berichtet die Ergebnisse: wie viele Tests gelaufen sind, wie viele bestanden (`ok`), fehlgeschlagen (`FAILED`), ignoriert oder gefiltert wurden.

- **Beispielausgabe:**

```
Compiling mein_projekt v0.1.0 (...)  
Finished test [unoptimized + debuginfo] target(s) in ...s  
Running target/debug/deps/mein_projekt-...
```

```
running 2 tests  
test tests::another_test ... ok  
test tests::it_works ... ok
```

```
test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in ...s
```

- **Nützliche `cargo test`-Optionen:**

- `cargo test <testname>`: Führt nur Tests aus, deren Name `<testname>` enthält.
- `cargo test -- --show-output`: Zeigt die `println!`-Ausgaben von bestandenen Tests an (standardmäßig unterdrückt).
- `cargo test -- --ignored`: Führt nur Tests aus, die mit `#[ignore]` markiert sind.
- `cargo test -- --test-threads=1`: Führt Tests nacheinander aus (nützlich bei Tests, die sich gegenseitig beeinflussen könnten).

- **23.4 Tests für `panic!`**

Manchmal möchten wir sicherstellen, dass unser Code unter bestimmten Fehlerbedingungen wie erwartet `panic!`t. Das `#[should_panic]`-Attribut über einer Testfunktion bewirkt, dass der Test *nur dann besteht*, wenn die Funktion tatsächlich `panic!`t.

- Rust

```
pub struct Guess {  
    value: i32,  
}  
  
impl Guess {  
    pub fn new(value: i32) -> Guess {  
        if value < 1 || value > 100 {  
            // Erwarteter Panic bei ungültigem Wert
```

```

        panic!("Guess value must be between 1 and 100, got {}.", value);
    }
    Guess { value }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    #[should_panic] // Dieser Test besteht nur, wenn Guess::new paniced
    fn greater_than_100() {
        Guess::new(200);
    }

    #[test]
    #[should_panic(expected = "must be between 1 and 100")] // Besteht nur, wenn
    // Panic-Meldung den Text enthält
    fn less_than_1() {
        Guess::new(0);
    }
}

```

-
-
- **23.5 Unit Tests vs. Integration Tests**

- **Unit Tests** (wie die oben gezeigten) testen typischerweise eine kleine Codeeinheit (z.B. eine Funktion oder ein Modul) isoliert. Sie werden oft im selben Modul wie der zu testende Code (in einem `#[cfg(test)] mod tests { ... }`) platziert und können auch private Funktionen testen.
- **Integration Tests** testen die öffentliche API Ihres Crates als Ganzes, oft so, wie ein externer Benutzer sie verwenden würde. Sie werden in separaten Dateien im `tests`-Verzeichnis auf der obersten Ebene Ihres Pakets (neben `src`) platziert. Jede Datei in `tests` wird als eigener kleiner Crate kompiliert, der Ihren Library Crate als Abhängigkeit verwendet. Sie können keine privaten Funktionen direkt testen.

Kapitel 24: Ein kleines Projekt

Nachdem wir nun eine breite Palette von Rusts Kernkonzepten abgedeckt haben, ist der beste Weg, dieses Wissen zu festigen, die Anwendung in einem praktischen Projekt.

- **Ziel:** Wenden Sie die Konzepte aus Teil 1 bis 5 an, um ein einfaches, aber vollständiges Programm zu erstellen.
- **Vorschläge:**
 1. **Zahlenratespiel:** Das Programm wählt eine Zufallszahl, der Benutzer muss sie erraten, das Programm gibt Hinweise ("zu hoch", "zu niedrig"). Benötigt: Zufallszahlen (`rand`-Crate), Benutzereingabe (`std::io`), String-Konvertierung (`parse`), Schleifen (`loop`), Bedingungen (`match` oder `if/else` auf `Ordering`), Fehlerbehandlung (`Result/expect`).
 2. **Einfacher Kommandozeilen-Taschenrechner:** Nimmt zwei Zahlen und einen Operator als Kommandozeilenargumente oder Eingabe entgegen und gibt das Ergebnis aus. Benötigt: Argument-Parsing (evtl. `std::env::args` oder `clap`-Crate), String-Konvertierung (`parse`), `match` für Operatoren, Funktionen, Fehlerbehandlung.
- **Mögliche Schritte:**
 1. **Setup:** Neues Projekt mit `cargo new mein_projekt`.
 2. **Design:** Überlegen Sie grob, welche Funktionen, Structs oder Enums Sie benötigen könnten.
 3. **Implementierung:** Schreiben Sie den Code Schritt für Schritt. Nutzen Sie `println!` zur Ausgabe und `std::io` oder externe Crates für Eingaben.
 4. **Fehlerbehandlung:** Verwenden Sie `Result` und `?` oder `match/expect` für Operationen, die fehlschlagen können (z.B. Parsen von Eingaben).
 5. **Organisation:** Teilen Sie den Code bei Bedarf in Module auf.
 6. **Abhängigkeiten:** Fügen Sie externe Crates (wie `rand`) über `Cargo.toml` hinzu.
 7. **Testen:** Schreiben Sie Unit Tests (`#[test]`) für Ihre Kernlogik.
- **Wichtiger Hinweis:** Das Ziel ist das Lernen und Anwenden. Es muss nicht perfekt sein! Experimentieren Sie, schauen Sie bei Bedarf in frühere Kapitel oder die offizielle Rust-Dokumentation (<https://doc.rust-lang.org/>) und haben Sie Spaß dabei, die Teile zusammenzufügen.

Herzlichen Glückwunsch! Sie haben nun eine Einführung in die wichtigsten Konzepte von Rust erhalten, von den Grundlagen über Ownership, Collections, Fehlerbehandlung bis hin zu Abstraktionen wie Generics, Traits und Lifetimes sowie dem Testen. Dies bildet eine solide Basis für Ihre weitere Reise mit Rust.

Natürlich gibt es noch viel mehr zu entdecken (Asynchronität, fortgeschrittene Traits und Typen, Makros, FFI, WebAssembly etc.), aber mit dem Wissen aus diesen fünf Teilen sind Sie gut gerüstet, um eigene Projekte zu starten und sich spezifische weiterführende Themen anzueignen.